COMPLETE STUDY GUIDE

Java / Spring Boot Technical Notes and Interview Preparation

Scope: Spring Cloud Gateway → Event-Driven Architecture → Tailwind CSS → WebSocket with STOMP → JUnit.

Learning sequence within each chapter: beginner foundations → core concepts → working flow → implementation → interview questions → debugging → revision.

Example baseline: Java 21, Maven, Spring Boot 3.5.16, Spring Cloud 2025.0.3 where needed, Spring Kafka 3.3 APIs, Tailwind CSS 4, and JUnit Jupiter 5. The Java projects are separate applications. Put each public Java type in its own file, under the package shown. Configuration alternatives are explicitly labelled; do not combine alternative route definitions.

This is a deliberate JUnit 5 teaching baseline, **not a claim that Boot 3.5 or JUnit 5 is the latest release or the recommended supported baseline for a new production deployment**. Boot 4 and JUnit 6 exist. JUnit 6 retains the Jupiter programming model and requires Java 17+. Boot 4 also changes some dependency modules, imports, and JSON APIs; do not mix examples from those generations. For deployment, select a currently supported Boot/Cloud combination and let its BOM manage dependency versions. Boot requirements, Cloud compatibility and support, Cloud 2025.0.3 release, JUnit 6 overview.

Examples are intentionally small. In-memory storage is labelled wherever used; it is not durable storage. Security extensions require a real trusted issuer. Kafka examples require a reachable broker. The document includes commands and expected results; execution verification is reported separately with the delivered guide.

Contents

Topic 1 - Spring Cloud Gateway 2

Topic 2 - Event-Driven Architecture 15

Topic 3 - Tailwind CSS 28

Topic 4 - WebSocket with STOMP 38

Topic 5 - JUnit 48

Final Comparison Tables 62

Topic 1 - Spring Cloud Gateway

1. What is it?

An **API gateway** is an entry point through which clients access backend APIs. It accepts a request, chooses a backend, applies shared policies, forwards the request, and returns the response. Spring Cloud Gateway implements this using Spring applications.

Suppose a shopping application has User Service and Product Service. Without a gateway, a browser must know both services' addresses. With a gateway, the browser calls one address, such as `http://localhost:8080`, and paths identify the service.

The problem it solves is repeated edge behavior: routing, authentication checks, rate limits, request transformation, and consistent access logging. It also hides internal service addresses and allows public paths to stay stable when backend locations change.

Analogy: a building receptionist directs visitors to departments and checks entry rules. The receptionist does not do every department's work.

Use it: when several APIs need a shared public entry point and centrally enforced edge policies; when routes or service instances change; when a Spring team needs custom Java filters.

Do not add it automatically: a single small API may be adequately exposed through an existing reverse proxy. A gateway adds a deployment, network hop, configuration, and availability responsibility. Keep business operations inside services.

Microservices do not technically require a gateway. It becomes useful when clients would otherwise coordinate many endpoints and duplicate cross-cutting policies.

These notes use **Gateway Server WebFlux**, built on WebFlux/Reactor with a Netty runtime. Gateway also has a Server Web MVC variant; its APIs and configuration are different. Do not infer that all Gateway variants are reactive. WebFlux starter and runtime.

2. Core Concepts

Term	Meaning	Why needed	Example
Route	A destination plus conditions, filters, identifier, and order	Defines one forwarding rule	`/api/users/**` goes to port 8081
Predicate	A condition evaluated against a request	Decides whether a route matches	`Path=/api/users/**`
Filter	Processing around forwarding	Changes or checks a request/response	Remove `/api` before forwarding
GatewayFilter	A filter attached to a route	Applies route-specific behavior	Product route adds a response header
GlobalFilter	A filter included in every matched gateway route's filter chain	Centralizes gateway-wide behavior	Record route latency
Filter factory	Creates configured filter instances	Lets YAML describe filter behavior	`StripPrefix=1`

Term	Meaning	Why needed	Example
Route matching	Evaluate ordered route candidates until a match	Chooses where this request goes	Lower route order is considered first
ServerWebExchange	Reactive request, response, and shared attributes	Carries state through the chain	Store the matched route
Mono	A reactive result with zero or one item	Represents asynchronous completion	`Mono<Void>` for filter completion
Request forwarding	Gateway issues an outbound request to the chosen URI	Connects edge request to backend	Netty HTTP client calls User Service
Load balancing	Choose an instance from several for one service	Spreads traffic and supports availability	`lb://user-service`
Service discovery	Obtain service addresses from a registry	Avoids hard-coding changing instances	Eureka lists User Service instances
CORS	Browser rules for reading cross-origin responses	Allows a permitted frontend origin to call APIs	Vite on port 5173 calls port 8080
Rate limit	Bound request usage for a key over time	Controls abuse and overload	10 requests/sec per authenticated user

Predicates select a route. Filters then act on its request and response. Discovery supplies instance addresses; load balancing selects one. A registry is not a request proxy.

Predicates you should know

The following are **individual examples**, to place under a route's `predicates:` list. Listing several predicates on one route normally means AND. Multiple methods inside one Method predicate mean GET **or** POST. YAML single quotes preserve regular-expression backslashes.

```
# Match a path tree; /api/users/42 is included.
- Path=/api/users/**
# Accept either of these methods.
- Method=GET,POST
# Require this header and a matching value.
- 'Header=X-Client-Version, v[0-9]+'
# Require a query parameter; regex is optional.
- Query=region
- Query=region, IN|US
# Evaluate the gateway server's current time, with an explicit zone.
- After=2026-10-01T00:00:00+05:30[Asia/Kolkata]
- Before=2026-11-01T00:00:00+05:30[Asia/Kolkata]
- Between=2026-10-01T00:00:00+05:30[Asia/Kolkata],2026-11-01T00:00:00+05:30[Asia/Kolkata]
```

`After` is after its instant; `Before` is before; `Between` is inside both boundaries. These conditions do not perform authentication and do not automatically create an alternative route outside the time window. Avoid inconsistent server clocks. A predicate mismatch commonly produces no matching route rather than a business validation response. Predicate reference.

Filters you should know

Filter	Effect	Input → output / example	Important caution
`StripPrefix=1`	Removes one leading path segment	`/api/users/42` → `/users/42`	Count segments, not characters

Filter	Effect	Input → output / example	Important caution
`RewritePath`	Regex replacement of path	`/api/users/42` → `/users/42`	YAML placeholder escaping matters
`AddRequestHeader=X-Edge, gateway`	Adds a header to outbound request	Backend sees `X-Edge`	Adds; it does not safely overwrite attacker identity
`AddResponseHeader=X-Served-By, gateway`	Adds a client-visible response header	Browser sees header	Avoid leaking infrastructure secrets
`SetRequestHeader`	Replaces the header value	Trusted identity header after validation	Only derive identity from verified credentials

An alternative to `StripPrefix=1` is:

```
filters:
  - 'RewritePath=/api/(?<remaining>.*), /$\{remaining}'
```

The named group captures everything after `/api/`. The replacement uses that capture with a leading slash. The `\` prevents Spring interpreting `${remaining}` as a configuration placeholder; it is not intended to remain in the forwarded path. Do not also strip the same prefix. RewritePath details.

3. Architecture / Working Flow

```
Browser / API client
       |
       v
Gateway HTTP server
       |
       +-- outer WebFilters, including Spring Security if configured
       |
       v
RoutePredicateHandlerMapping: find a matching route
       |
       v
FilteringWebHandler: combine GatewayFilters + GlobalFilters
       |
       v
Pre-processing --> resolve destination --> outbound HTTP request
                                           |
                                           v
                                 User OR Product Service
                                           |
       client <-- response writing <-- post-processing
```

1. Netty receives a request. An optional security WebFilter can reject invalid credentials before route handling.
2. The handler mapping evaluates route predicates against the incoming request. Filters such as `StripPrefix` do not normally rewrite the path before this match.
3. Gateway combines global and route-specific filters and sorts the chain. Lower order values execute earlier on the way in; surrounding post-processing unwinds in reverse order.
4. Pre-filters can modify headers/path, enforce limits, or stop the chain.
5. Destination resolution forms the outbound URL. An `lb://` destination requires service-instance resolution.
6. The HTTP client forwards the request. The backend executes its normal controller/service work.
7. Response processing can add headers or record latency. Gateway sends status/body back to the client.

Important: a `GlobalFilter` is global to matched gateway routes. It is not a replacement for security WebFilters covering unmatched requests, preflight handling, or other application endpoints. Filter order must be understood relative to Gateway's built-in routing/response filters. Gateway working model.

4. Practical Implementation

A. Three applications and their paths

```
                  Gateway :8080
Client ----------------+------------------+
 /api/users/42          |                  | /api/products/7
                        v                  v
               User Service :8081   Product Service :8082
                  /users/42            /products/7
```

This is the usual gateway architecture. The requested `Client → Gateway → User Service → Product Service` chain additionally requires User Service to call Product Service for a particular business operation. Routing configuration alone does not create that service-to-service call. Sections B-G demonstrate independent routing; the small extension below also implements the requested chain.

B. Maven setup

Use this complete minimal Gateway `pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>
  <groupId>com.example</groupId>
  <artifactId>gateway-demo</artifactId>
  <version>1.0.0</version>
  <properties><java.version>21</java.version></properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>2025.0.3</version>
        <type>pom</type><scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
    </dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
  </plugin></plugins></build>
</project>
```

The parent manages Boot dependencies and compiler settings. `java.version` selects Java 21. The Cloud BOM aligns Cloud libraries with the selected release train. The starter provides Gateway WebFlux; the plugin supports `mvn spring-boot:run` and executable jars. Do not add `spring-boot-starter-web` to this reactive gateway.

For **each backend**, reuse the parent, Java property, and plugin, remove the Cloud BOM, change `artifactId`, and replace the dependencies with `org.springframework.boot:spring-boot-starter-web`. Their controllers use MVC and may run independently of the reactive gateway.

C. Boot entry point

```
package com.example.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }
}
```

`@SpringBootApplication` enables configuration, auto-configuration, and component scanning from this package downward. `run` creates the application context and starts the server. Create equivalent entry classes in `com.example.users` and `com.example.products` for the two backends. Keep their controllers under those roots so scanning finds them.

D. Gateway application.yml

Put this in `src/main/resources/application.yml`:

```
server:
  port: 8080
spring:
  application:
    name: gateway-demo
  cloud:
    gateway:
      server:
        webflux:
          httpclient:
            connect-timeout: 2000
            response-timeout: 5s
          routes:
            - id: users-route
              uri: http://localhost:8081
              predicates:
                - Path=/api/users/**
              filters:
                - StripPrefix=1
                - AddRequestHeader=X-Edge, gateway-demo
                - AddResponseHeader=X-Served-By, gateway-demo
            - id: products-route
              uri: http://localhost:8082
              predicates:
                - Path=/api/products/**
              filters:
                - StripPrefix=1
          globalcors:
            add-to-simple-url-handler-mapping: true
            cors-configurations:
              '[/**]':
                allowedOrigins:
                  - http://localhost:5173
                allowedMethods: [GET, POST, DELETE, OPTIONS]
                allowedHeaders: [Authorization, Content-Type]
                exposedHeaders: [X-Served-By]
                maxAge: 3600
```

Read it line by line:

- Port 8080 is the public development entry point; `name` identifies the application.
- `server.webflux` selects the modern WebFlux-specific property namespace. Older examples often use `spring.cloud.gateway.routes`; check your release's binding metadata before copying them.
- `connect-timeout` is in milliseconds; `response-timeout` is a duration. They limit two different waits.
- Each `id` uniquely names a route. `uri` identifies its backend.
- `Path` tests the original incoming path. `StripPrefix=1` changes it only after selection.
- Header filters demonstrate request/response modification. They are not authentication.
- CORS allows a specific frontend origin. `OPTIONS` preflight can be handled even if a method predicate prevents route matching, using the simple-handler option.
- `exposedHeaders` allows browser JavaScript to read this non-safelisted response header; it does not create the header.

This is local HTTP configuration. Use HTTPS at the public edge in production. Credentialed CORS must allow explicit origins; do not combine wildcard origins with credentials. Prefer one consistent CORS policy owner to avoid duplicate headers. Property names, CORS options.

E. Backend controllers

User Service has `server.port=8081` in `application.properties`:

```
package com.example.users;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {
    public record UserView(long id, String name) {}

    @GetMapping("/users/{id}")
    public UserView get(@PathVariable("id") long id) {
        return new UserView(id, "Ravi");
    }
}
```

Product Service has `server.port=8082`:

```
package com.example.products;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ProductController {
    public record ProductView(long id, String name) {}

    @GetMapping("/products/{id}")
    public ProductView get(@PathVariable("id") long id) {
        return new ProductView(id, "Keyboard");
    }
}
```

`@RestController` registers HTTP handlers whose return values become JSON. `@GetMapping` binds a GET path. `@PathVariable` converts the path segment to `long`. Records are concise immutable response models. These controllers return fixed demo data, so they test routing rather than persistence.

Start each application in its own terminal with `mvn spring-boot:run`. Test with PowerShell:

```
Invoke-RestMethod 'http://localhost:8080/api/users/42'
Invoke-RestMethod 'http://localhost:8080/api/products/7'
```

Expected results: user `{id:42,name:"Ravi"}` and product `{id:7,name:"Keyboard"}`. A direct backend call uses `/users/42`, while a gateway call uses `/api/users/42`. If the backend works directly and the gateway returns 404, inspect route matching and the transformed path first.

Optional chain using the same services: add `UserProductController.java` to User Service:

```
package com.example.users;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestClient;

@RestController
public class UserProductController {
    private final RestClient products;

    public UserProductController(RestClient.Builder builder) {
        this.products = builder.baseUrl("http://localhost:8082").build();
    }

    public record ProductView(long id, String name) {}
    public record UserProductView(long userId, ProductView featuredProduct) {}

    @GetMapping("/users/{id}/featured-product")
    public UserProductView featured(@PathVariable("id") long id) {
        ProductView product = products.get().uri("/products/7")
            .retrieve().body(ProductView.class);
        return new UserProductView(id, product);
    }
}
```

Boot supplies the `RestClient.Builder` in this MVC backend. The constructor sets Product Service's base address. `get().uri(...).retrieve().body(...)` performs a synchronous HTTP GET and converts its JSON to the local record. The response combines the user's requested ID with that product. This is a minimal demonstration of cooperation, not a recommendation algorithm.

Calling `/api/users/42/featured-product` at Gateway now follows `Client → Gateway → User Service → Product Service`; the response returns through User Service and Gateway. The existing user route already matches it. Product unavailability now also affects this User Service operation. Configure bounded downstream timeouts and deliberate error handling in a real backend. This blocking client belongs in the MVC service, not in a WebFlux gateway filter.

F. Equivalent Java route configuration

Use this **instead of** the YAML `routes` list. Leave other YAML settings intact.

```
package com.example.gateway;

import org.springframework.cloud.gateway.route.RouteLocator;
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class GatewayRoutes {
    @Bean
    RouteLocator routes(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("users-route", route -> route.path("/api/users/**")
                .filters(filters -> filters.stripPrefix(1)
                    .addRequestHeader("X-Edge", "gateway-demo")
                    .addResponseHeader("X-Served-By", "gateway-demo"))
                .uri("http://localhost:8081"))
            .route("products-route", route -> route.path("/api/products/**")
                .filters(filters -> filters.stripPrefix(1))
                .uri("http://localhost:8082"))
```

```
            .build();
    }
}
```

`@Configuration` contributes bean definitions; `@Bean` registers the returned `RouteLocator`. The builder creates the same two routes. Java is useful for custom predicates or filters; YAML is convenient for declarative configuration. Duplicating both definitions causes confusing route selection.

G. Global logging filter

```
package com.example.gateway;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component
public class TimingFilter implements GlobalFilter, Ordered {
    private static final Logger log = LoggerFactory.getLogger(TimingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        return Mono.defer(() -> {
            long started = System.nanoTime();
            return chain.filter(exchange).doFinally(signal ->
                log.info("requestId={} method={} path={} status={} elapsedMs={} signal={}",
                    exchange.getRequest().getId(),
                    exchange.getRequest().getMethod(),
                    exchange.getRequest().getPath(),
                    exchange.getResponse().getStatusCode(),
                    (System.nanoTime() - started) / 1_000_000, signal));
        });
    }

    @Override
    public int getOrder() { return -100; }
}
```

`@Component` makes the filter discoverable. `chain.filter` continues processing; returning it connects asynchronous work to the request lifecycle. `Mono.defer` starts the clock when subscribed. `doFinally` logs completion, cancellation, or error. `getOrder` places this filter relative to other filters; `-100` is a chosen demonstration order, not a universally correct order.

The status may be absent on cancellation or errors before response handling. Do not log Authorization headers, tokens, private bodies, or sensitive query parameters. Do not call `.block()`, perform JDBC calls, or sleep on an event-loop thread. A reactive gateway can still be slowed by blocking custom filters.

H. Authentication and JWT validation extension

Add `org.springframework.boot:spring-boot-starter-oauth2-resource-server` to Gateway dependencies. Configure a **real trusted issuer**, not this illustrative hostname:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://identity.example.com/issuer
          audiences: [shop-api]
```

Merge this into the existing `spring` mapping; do not create duplicate YAML keys.

```
package com.example.gateway;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;

@Configuration
public class GatewaySecurity {
    @Bean
    SecurityWebFilterChain apiSecurity(ServerHttpSecurity http) {
        return http
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .authorizeExchange(auth -> auth
                .pathMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .pathMatchers("/api/**").authenticated()
                .anyExchange().denyAll())
            .oauth2ResourceServer(resource -> resource.jwt(Customizer.withDefaults()))
            .build();
    }
}
```

This extension assumes a stateless API where browsers explicitly send bearer tokens, rather than browser-managed cookie authentication; that assumption is why CSRF is disabled. Keep CSRF protection when credentials are automatically attached by the browser. Preflight is permitted; API paths require authentication; other paths are rejected. Spring Security handles JWT authentication through a WebFilter chain.

A JWT contains a header, claims, and signature. Base64-decoding it is **not validation**. Verify the signature with trusted keys and accepted algorithms; validate issuer, expiration, not-before, and audience. Authentication establishes identity. Authorization still decides whether that identity may access an operation or object. Missing/invalid credentials normally produce 401; insufficient permission produces 403. Reactive JWT support, Boot audience property.

Backend services must still enforce business authorization. Prevent direct public access to internal services; do not trust arbitrary client-supplied `X-User-Id`. If forwarding derived identity, remove or overwrite spoofable incoming headers and protect the trust boundary. Never invent a JWT validator that merely checks whether an Authorization header exists.

I. Rate limiting, load balancing, and Eureka

Rate limiting: a token bucket refills tokens at a configured rate; each request consumes tokens. A burst capacity allows short spikes. The Redis-based Gateway limiter uses shared state, so multiple gateway instances can enforce a common limit. It requires `spring-boot-starter-data-redis-reactive`, reachable Redis, and a rate-limit key.

Define this bean when authentication is enabled:

```
@Bean
org.springframework.cloud.gateway.filter.ratelimit.KeyResolver userKeyResolver() {
    return exchange -> exchange.getPrincipal().map(java.security.Principal::getName);
}
```

It resolves a trusted authenticated identity. Add this filter to a protected route:

```
- name: RequestRateLimiter
  args:
    key-resolver: '#{@userKeyResolver}'
    redis-rate-limiter.replenishRate: 10
    redis-rate-limiter.burstCapacity: 20
    redis-rate-limiter.requestedTokens: 1
```

10 tokens/sec refill, 20-token bucket capacity, and one token/request produce a per-user budget. Excess traffic receives 429 by default. Decide what happens when Redis is unavailable and when the key is missing; an empty

key is denied by default. An in-memory limit on each gateway does not produce a fleet-wide limit. Never trust a raw user-id header or unvalidated forwarded IP as a limiter identity.

Discovery and balancing: add `org.springframework.cloud:spring-cloud-starter-netflix-eureka-client` and `org.springframework.cloud:spring-cloud-starter-loadbalancer` to Gateway. Register the services with Eureka using their own client dependency, names, and registry URL. A separately running Eureka registry is assumed here; its administration is outside this topic.

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Name the backend applications `user-service` and `product-service`. Replace the route URIs with `lb://user-service` and `lb://product-service`. The discovery client gets eligible instances; Spring Cloud LoadBalancer selects one; Gateway forwards to its host/port. No eligible instance normally gives 503, subject to configuration.

Explicit `lb://` routes do not require the discovery locator. The optional `spring.cloud.gateway.server.webflux.discovery.locator.enabled=true` generates discovery-based routes automatically. Review its inclusion rules carefully so internal services are not unintentionally exposed. Eureka registration is eventually refreshed and can contain stale instances; handle connection failures and health transitions.

J. Error handling and security in practice

Symptom	Meaning to investigate	Action
404	No route matched, or backend returned 404	Compare original path, route predicates, and forwarded path
401 / 403	Authentication / authorization rejection	Inspect trusted issuer, claims, audience, and access policy
429	Rate budget exceeded	Inspect key, refill, burst, and legitimate usage
503	Service unavailable; e.g. no discovered instance	Check registry and backend readiness
Timeout / connection error	Backend slow or unreachable	Check DNS, port, timeout, and backend latency
Browser-only CORS error	Browser rejects cross-origin interaction	Check preflight and actual response headers

Distinguish a backend's HTTP error response from a transport exception before any response exists. Exact transport-error status mapping depends on the error handler; do not assume every network failure becomes 502 automatically. A reactive `ErrorWebExceptionHandler` can produce a consistent sanitized JSON error envelope. Security rejection handlers are configured separately in Spring Security. Never return a fake success for a failed operation.

Log request/correlation identifier, route identifier, response status, elapsed time, and sanitized exception category. Set connect/response/body-size limits. Protect gateway management endpoints. Permit only reviewed destinations to avoid open-proxy behavior. Restrict trusted proxy headers. Keep gateway instances redundant behind an entry load balancer. Retry reads selectively; retrying a payment POST without an idempotency strategy can duplicate the payment.

Important Interview Questions

10 beginner questions

1. **What is an API gateway?** A shared API entry point that routes requests and applies edge policies before returning backend responses.
2. **What is Spring Cloud Gateway?** Spring's gateway framework with route predicates and filters; this example uses its reactive Server WebFlux variant.
3. **Why use a gateway with microservices?** To give clients stable public APIs and centralize routing and shared access policies.
4. **What is a route?** A named destination with matching predicates, filters, and order.
5. **What is a predicate?** A boolean decision determining whether a route handles a request.
6. **What is a filter?** Processing that surrounds forwarding, such as path rewriting or header changes.
7. **What does StripPrefix do?** Removes a configured number of leading path segments before forwarding.
8. **Does the gateway replace backend controllers?** No. Backends still implement business operations and authorization.
9. **What is CORS?** Browser-enforced rules controlling access to responses across origins; it is not authentication.
10. **What is load balancing?** Selecting one instance among several instances of the same service.

10 intermediate questions

1. **GatewayFilter vs GlobalFilter?** A GatewayFilter is attached to a route; a GlobalFilter participates in the chain for all matched gateway routes.
2. **How are multiple predicates combined?** Normally AND across predicates on a route; alternatives within one predicate may use OR.
3. **When does path rewriting occur?** After matching the route against the incoming request, before forwarding.
4. **How does filter ordering work?** Lower order executes earlier in pre-processing; surrounding post-processing unwinds in reverse.
5. **Why avoid blocking in WebFlux Gateway?** A small event-loop pool serves many connections; blocking a thread delays other requests.
6. **How does lb:// work?** Discovery supplies service instances, and a load-balancer filter chooses the target address.
7. **Is Eureka the load balancer?** No. Eureka is the registry; the load balancer selects from discovered instances.
8. **How should JWT authentication be implemented?** Use Spring Security resource-server support with trusted signature and claim validation.
9. **Why use a shared rate-limit store?** To coordinate limits across several gateway instances.
10. **Gateway vs Nginx?** Both can reverse-proxy. Gateway suits Spring-integrated Java policies; Nginx is a general web server/proxy with overlapping routing, balancing, and other capabilities.

5 scenario-based questions

1. **The gateway goes down. What happens?** Clients using that entry cannot reach APIs even if backends are healthy. Run multiple gateway instances with health-aware traffic distribution.
2. **Direct /users/42 works but /api/users/42 fails. What do you check?** Gateway route registration, Path match, backend URI, StripPrefix, and whether 404 came from Gateway or backend.

3. **Requests exceed a user's budget. What should happen?** Return 429 using a trusted per-user key and a shared limit; provide useful retry guidance without retry storms.
4. **An expired JWT is decoded successfully. Is it valid?** No. Decoding only reads claims; signature and time/issuer/audience validation must pass.
5. **A client forges X-User-Id. How do you prevent impersonation?** Ignore or replace that header with verified identity and ensure backends only trust identity from protected paths.

Common Mistakes

- **Using the MVC starter in a WebFlux gateway:** select one Gateway runtime; keep backend MVC dependencies in backend applications.
- **Mismatching Boot and Cloud versions:** use the compatibility table and BOM; do not suppress compatibility checks to hide the mismatch.
- **Wrong property namespace:** inspect your exact Gateway release's metadata; this guide uses `server.webflux`.
- **Duplicate YAML keys:** merge nested mappings; a second `spring:` may overwrite or fail parsing.
- **Treating predicates as filters:** a failed predicate does not transform the request or perform business validation.
- **Stripping twice:** use either the intended StripPrefix or RewritePath transformation, then verify the outgoing path.
- **JWT presence checks:** validate cryptography and claims through resource-server support.
- **Blocking and sensitive logs:** keep filters nonblocking and log only sanitized request metadata.
- **Putting business workflows in Gateway:** keep gateway responsibilities focused on edge traffic policy.
- **Assuming CORS protects an API from all clients:** nonbrowser clients are not stopped by browser CORS rules; enforce authentication and authorization.

Quick Revision

- Route = destination + conditions + filters.
- Predicate chooses; filter processes.
- Route matching uses the incoming path.
- GatewayFilter is route-specific; GlobalFilter covers matched gateway routes.
- WebFlux Gateway uses a reactive chain and Netty; MVC Gateway is a separate variant.
- JWT decoding differs from validation.
- Discovery finds instances; load balancing selects one.
- Diagnose original path, forwarded path, route ID, and backend status separately.

Remember This for Interviews

1. “An API gateway centralizes edge policies while services own business rules.”
2. “A route predicate decides whether a request matches; a filter can change processing.”
3. “GlobalFilter does not mean every possible application request is secured.”
4. “Nonblocking I/O helps concurrency only when custom processing also avoids blocking.”
5. “Service discovery and load balancing solve different parts of destination selection.”
6. “A JWT is trusted only after its signature and required claims are validated.”
7. “Rate limits and availability must be designed for multiple gateway instances.”
8. “An extra network hop means Gateway needs its own timeouts, capacity, and redundancy.”

Topic 2 - Event-Driven Architecture

1. What is it?

Event-Driven Architecture (EDA) organizes communication around facts that happened. A service publishes an event such as `OrderCreated`; other services react when they receive it.

An **event** describes an occurrence, usually in past tense. `OrderCreated` is a fact. `ChargeCustomer` is a command requesting an action. Both may travel through a message broker, but their meanings differ.

EDA exists because a business action often interests several independent systems. Order creation may trigger payment, stock processing, and a confirmation message. Calling every system directly ties order handling to each system's location, latency, and availability.

Analogy: a university posts an exam-result announcement. Scholarship staff and students independently react. The announcing department does not phone every interested person in sequence.

Use it: when several independent consumers react to changes; when work may finish later; when buffering, replay, and independent scaling are useful.

Do not use it automatically: a simple operation needing an immediate authoritative answer may be clearer as a synchronous call. EDA adds delivery failures, schema contracts, monitoring, and consistency decisions. Do not promise instant global consistency when the design processes events asynchronously.

In a Spring Boot application, a producer sends records through a Kafka client and a listener container invokes consumer methods. Kafka is the broker used here; EDA itself is not limited to Kafka.

2. Core Concepts

Term	Meaning	Why needed	Example
Event	A fact about an occurrence	Communicates what happened	`OrderCreated`
Producer / publisher	Component that emits an event	Makes facts available beyond its boundary	Order Service sends a record
Consumer	Component that reads and processes records	Performs reactions	Inventory Service reserves stock
Subscriber	A consumer interested in a channel	Expresses subscription interest	Notification app listens to orders
Event broker	Intermediary storing/routing events	Decouples producer availability from consumers	Kafka brokers
Event channel	Logical place where events are published	Organizes subscriptions	Topic `orders.created.v1`
Event stream	Ordered sequence of records over time	Supports ongoing processing/replay	Records across an order topic
Partition	Ordered log within a Kafka topic	Splits storage and parallel consumption	Orders with a chosen key share a partition
Offset	Position of a record in a partition	Tracks progress and supports replay	Partition 0, offset 120

Term	Meaning	Why needed	Example
Consumer group	Consumers coordinating one logical subscription	Shares work across instances	Three payment instances use `payments`
Asynchronous communication	Sender need not wait for business processing	Removes direct consumer latency from the request	Payment happens after order acceptance
Loose coupling	Less direct knowledge of other components	Allows independent evolution/deployment	Producer knows event contract, not subscriber URLs
Eventual consistency	Related states converge after processing	Reflects asynchronous updates honestly	Order accepted now, payment status later
Idempotency	Repeating an operation has the intended effect only once	Makes redelivery safe	Same payment event does not charge twice
Duplicate event	A repeated delivery or repeated publication	Forces deduplication and business safeguards	Same `eventId` appears again
Event ordering	Sequence in which related facts are observed	Avoids invalid state transitions	Created before Cancelled
Retry	Reattempt after failure	Recovers transient problems	Temporary payment timeout
Dead Letter Queue / Topic	Destination for records that cannot be processed normally	Preserves failures for investigation/recovery	`orders.created.v1.DLT`
Acknowledgement	Confirmation at a particular boundary	Determines delivery/progress guarantees	Producer broker ACK; consumer offset commit
Event schema	Agreed structure/types/meaning	Allows independent readers to understand events	`eventId`, `orderId`, `amount`, `currency`
Event versioning	Manage schema and semantic changes over time	Prevents deployments breaking old consumers	Add optional field vs introduce a v2 contract

A broker ACK is not a consumer ACK. Producer acknowledgement says something about broker acceptance/durability under configuration. Consumer acknowledgement/offset commit records consumption progress. Neither alone proves a business transaction or external payment succeeded.

Notification, carried state, and processing

- **Event notification:** publish `OrderCreated(orderId)`. A consumer fetches details if needed. Small payload, but consumers depend on the source API and may see its later state.
- **Event-carried state transfer:** include fields needed by consumers, such as order amount, currency, and items. Consumers can act without immediate calls back, but data is duplicated and schemas need care.
- **Event processing:** consume events, filter/transform them, update state, or derive new facts. Payment may publish `PaymentAuthorized` or `PaymentFailed`.

The example carries enough order state for the three demo consumers. Avoid publishing unnecessary personal data or secrets simply because events are convenient.

Kafka ordering, groups, and delivery

Kafka preserves record order **within a partition**, not a global total order across a topic. Choose a consistent key such as `orderId` when related events must reach the same partition. Keying does not correct records a producer publishes in the wrong business order. Increasing partition count can change future key placement, so strict per-entity ordering needs migration planning.

Within a group, a partition is assigned to at most one active consumer at a time. Different groups independently read the same topic. If payment, inventory, and notifications use **one shared group**, they divide records; they do not all receive each order. Use distinct group IDs for independent reactions.

At-most-once: mark progress before effect; a crash can lose work. **At-least-once:** effect then commit progress; a crash between them can repeat work. **Exactly-once:** a scoped guarantee requiring coordinated transactions; Kafka's transactional processing does not automatically make external databases, emails, or payment APIs execute once. Producer idempotence prevents particular retry duplicates at the broker, not all duplicate business events. Kafka delivery and log design.

3. Architecture / Working Flow

Independent reactions

```
Client creates an order
          |
          v
     Order Service
          |
          v
 OrderCreated event -> Kafka topic: orders.created.v1
                              |
                +-------------+----------------+
                v             v                v
          Payment group  Inventory group  Notification group
```

1. Order Service validates and records an order.
2. It constructs an event with a stable identifier, order ID, occurrence time, version, and required business data.
3. A producer serializes it to JSON and sends it to a broker using `orderId` as the key.
4. Kafka appends the record to a partition and acknowledges under the producer configuration.
5. Each group independently consumes the record. Its listener deserializes JSON and performs its reaction.
6. Each group records progress when processing/recovery succeeds. Other groups may be ahead or behind.

If payment must precede inventory

```
Order Service --OrderCreated--> Kafka --> Payment Service
Payment Service --PaymentAuthorized--> Kafka --> Inventory Service
Inventory Service --InventoryReserved--> Kafka --> Notification Service
```

This is the requested business sequence made explicit. Subscribing all three services to `OrderCreated` does **not** enforce that sequence. Failure events also need defined reactions: for example, payment failure should not result in an “order completed” notification. The code below demonstrates independent subscriptions, while this diagram explains sequential event dependencies.

Synchronous vs event-driven interaction

In a synchronous call, Order Service waits for a result from another service; a failure directly affects the current call. In EDA, it can acknowledge local acceptance before downstream processing finishes. This improves independence but moves completion/failure reporting into status updates or later events. “Asynchronous” does not mean “no latency,” “no coupling,” or “no failure.” The event contract remains a form of coupling.

4. Practical Implementation

A. Project setup

Create an `orders-events-demo` application with the same Boot parent, Java property, and Maven plugin used in Chapter 1. **No Cloud BOM or Gateway dependency is needed.** Dependencies:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>
</dependencies>
```

Web exposes a small HTTP trigger. Validation rejects invalid requests. Spring Kafka supplies producer templates, listener containers, serializers, and error handlers. Boot manages dependency versions. Create `EventsApplication` in `com.example.events` with the same `@SpringBootApplication` and `SpringApplication.run` pattern.

The example runs producer and three logically separate consumers in one process for learning. A real deployment separates the services; each uses the same wire contract and its own group ID. Broker connectivity is still real, not an in-process substitute.

B. Start a local broker

With Docker installed, the following official Apache Kafka image gives a single-node local broker:

```
docker run --name notes-kafka -p 9092:9092 -d apache/kafka:4.1.0
```

Wait until Kafka is ready before starting the application. This is a local single-node example, not a production cluster. `localhost:9092` works when the application runs on the host. If the application is also containerized, its network address and Kafka's advertised listeners must be configured accordingly. A broker bootstrap address being reachable is insufficient if the addresses returned in metadata are not reachable. Apache Kafka Docker image.

C. Event and request models

`OrderCreatedEvent.java`:

```
package com.example.events;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

public record OrderCreatedEvent(
    UUID eventId,
    int schemaVersion,
    Instant occurredAt,
    String orderId,
    BigDecimal amount,
    String currency
) {}
```

`CreateOrderRequest.java`:

```
package com.example.events;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import java.math.BigDecimal;

public record CreateOrderRequest(
    @NotBlank String orderId,
    @NotNull @DecimalMin("0.01") BigDecimal amount,
    @NotBlank @Pattern(regexp = "[A-Z]{3}") String currency
) {}
```

`eventId` identifies one event independently of the order. `schemaVersion` identifies the contract generation. `occurredAt` is the occurrence time, distinct from broker/consumer receipt time. `orderId` links business facts and becomes the partition key. `BigDecimal` avoids binary floating-point error for decimal amounts. `currency` makes an amount meaningful; the regex checks form, not whether a currency is an accepted business currency.

Records supply constructors/accessors without mutable setters. Request validation checks positive amount and required fields; event records still require consumer contract validation because broker input is not guaranteed to come from this HTTP controller.

D. Kafka application.yml

```
server:
  port: 8083
spring:
  application:
    name: orders-events-demo
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      properties:
        enable.idempotence: true
        spring.json.add.type.headers: false
    consumer:
      enable-auto-commit: false
      auto-offset-reset: earliest
      key-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.key.delegate.class: org.apache.kafka.common.serialization.StringDeserializer
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.value.default.type: com.example.events.OrderCreatedEvent
        spring.json.trusted.packages: com.example.events
        spring.json.use.type.headers: false
    listener:
      ack-mode: record
```

Read it line by line:

- `bootstrap-servers` seeds cluster discovery; Kafka then supplies broker metadata.
- String keys and JSON values become bytes on publication.
- `acks: all` waits for the configured in-sync-replica acknowledgement condition. Actual durability also depends on replication and minimum in-sync replicas; this single-node lab has no replica redundancy.
- Producer idempotence protects Kafka retry publication under its guarantees. It does not identify two distinct sends representing the same order.
- Disabling Java type headers keeps the wire contract independent of Java package names. Consumers explicitly select their local target type.

- `ErrorHandlingDeserializer` wraps deserialization failures so the container can recover them. They occur before your listener method executes.
- Restrict trusted packages; avoid `*`. A fixed target type plus disabled type headers limits reliance on producer-supplied class identity.
- Auto-commit is disabled; `ack-mode: record` lets the container commit after a record listener returns successfully or after successful configured recovery.
- `earliest` is used only when the group lacks a valid committed offset; it does not reread the topic from the beginning on every restart.

JSON example: `{"eventId":"...","schemaVersion":1,"occurredAt":"2026-10-04T10:00:00Z","orderId":"O-100","amount":499.00,"currency":"INR"}`. The UUID and timestamp are JSON strings; amount is a numeric value. JSON encoding and decoding are handled by the configured serializers. These are the Spring Kafka 3.3 Jackson 2 APIs; newer generations have different Jackson-related choices. Spring Kafka JSON and deserialization handling.

E. Topic configuration

`KafkaTopics.java`:

```
package com.example.events;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopics {
    public static final String ORDERS = "orders.created.v1";

    @Bean
    NewTopic ordersTopic() {
        return TopicBuilder.name(ORDERS).partitions(3).replicas(1).build();
    }

    @Bean
    NewTopic deadLetterTopic() {
        return TopicBuilder.name(ORDERS + ".DLT").partitions(3).replicas(1).build();
    }
}
```

`NewTopic` beans let Boot's Kafka admin create local topics. Three partitions allow up to three actively assigned consumer instances per group for this topic. Replicas=1 matches the one-broker lab. The DLT has at least the same partition count because default recovery republishes to the source partition number. In production, provision topics explicitly with reviewed replication, retention, and ACLs.

F. Producer and HTTP trigger

`OrderEventProducer.java`:

```
package com.example.events;

import java.time.Instant;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderEventProducer {
    private final KafkaTemplate<String, OrderCreatedEvent> kafka;

    public OrderEventProducer(KafkaTemplate<String, OrderCreatedEvent> kafka) {
```

```
        this.kafka = kafka;
    }

    public CompletableFuture<OrderCreatedEvent> publish(CreateOrderRequest request) {
        var event = new OrderCreatedEvent(UUID.randomUUID(), 1, Instant.now(),
            request.orderId(), request.amount(), request.currency());
        return kafka.send(KafkaTopics.ORDERS, event.orderId(), event)
            .thenApply(result -> event);
    }
}
```

`OrderEventController.java`:

```
package com.example.events;

import jakarta.validation.Valid;
import java.util.concurrent.CompletableFuture;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/demo/order-events")
public class OrderEventController {
    private final OrderEventProducer producer;

    public OrderEventController(OrderEventProducer producer) {
        this.producer = producer;
    }

    @PostMapping
    public CompletableFuture<ResponseEntity<OrderCreatedEvent>> publish(
            @Valid @RequestBody CreateOrderRequest request) {
        return producer.publish(request).thenApply(event ->
            ResponseEntity.accepted().body(event));
    }
}
```

`@Service` registers the producer; constructor injection supplies the template. `send(topic,key,value)` chooses the topic and partition key, serializes the event, and returns a future. `thenApply` runs after producer completion; it does not wait for downstream business processing. `@Valid` triggers request validation. The controller returns 202 after broker-send success, meaning downstream work may still be pending.

This endpoint is explicitly a **demo event trigger**: it assumes an order was created and publishes its fact. It does not persist an order or charge money. In a real Order Service, publish only after the associated business operation is recorded using the reliability strategy below.

Do not discard the send future and immediately claim publication success. Handle failed publication with a useful sanitized error/status. A timeout can have an uncertain outcome: the record may have been accepted although the client did not observe confirmation. Retrying HTTP requests must use a business idempotency key or stable event record; a new random event ID on every user retry is not business deduplication.

G. Consumers

`OrderEventConsumers.java`:

```
package com.example.events;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderEventConsumers {
```

```
    private static final Logger log = LoggerFactory.getLogger(OrderEventConsumers.class);

    @KafkaListener(topics = KafkaTopics.ORDERS, groupId = "payments-demo")
    public void payment(OrderCreatedEvent event) {
        validate(event);
        log.info("Payment reaction eventId={} orderId={} amount={} currency={}",
            event.eventId(), event.orderId(), event.amount(), event.currency());
    }

    @KafkaListener(topics = KafkaTopics.ORDERS, groupId = "inventory-demo")
    public void inventory(OrderCreatedEvent event) {
        validate(event);
        log.info("Inventory reaction eventId={} orderId={}",
            event.eventId(), event.orderId());
    }

    @KafkaListener(topics = KafkaTopics.ORDERS, groupId = "notifications-demo")
    public void notification(OrderCreatedEvent event) {
        validate(event);
        log.info("Notification reaction eventId={} orderId={}",
            event.eventId(), event.orderId());
    }

    private static void validate(OrderCreatedEvent event) {
        if (event == null || event.schemaVersion() != 1 || event.eventId() == null
                || event.occurredAt() == null || event.orderId() == null
                || event.orderId().isBlank() || event.amount() == null
                || event.amount().signum() <= 0 || event.currency() == null
                || !event.currency().matches("[A-Z]{3}")) {
            throw new IllegalArgumentException("Invalid order event contract");
        }
    }
}
```

`@KafkaListener` creates a listener endpoint backed by a managed container; each distinct group reads independently. Boot configures Kafka listeners, so a separate `@EnableKafka` is not needed for this Boot example. Each method receives the deserialized record value. Returning successfully allows configured progress recording; throwing enters error handling.

The demo logs reactions so delivery is visible. Real payment/inventory/email effects replace those log lines and need idempotency. These methods have no meaningful ordering across groups. Do not swallow an exception and return success: that can commit the offset despite failed business processing.

H. Retry and DLT configuration

`KafkaFailureConfig.java`:

```
package com.example.events;

import jakarta.annotation.PreDestroy;
import java.util.HashMap;
import java.util.Map;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.DefaultKafkaProducerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.listener.DeadLetterPublishingRecoverer;
import org.springframework.kafka.listener.DefaultErrorHandler;
import org.springframework.kafka.support.serializer.DelegatingByTypeSerializer;
import org.springframework.kafka.support.serializer.JsonSerializer;
import org.springframework.util.backoff.FixedBackOff;

@Configuration
public class KafkaFailureConfig {
```

```
    private DefaultKafkaProducerFactory<Object, Object> recoveryFactory;

    @Bean
    DefaultErrorHandler kafkaErrorHandler(
            @Value("${spring.kafka.bootstrap-servers}") String brokers) {
        var properties = new HashMap<String, Object>();
        properties.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, brokers);
        properties.put(ProducerConfig.ACKS_CONFIG, "all");
        properties.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

        var json = new JsonSerializer<OrderCreatedEvent>();
        json.setAddTypeInfo(false);
        var values = new DelegatingByTypeSerializer(Map.of(
            byte[].class, new ByteArraySerializer(),
            OrderCreatedEvent.class, json));

        var keys = new DelegatingByTypeSerializer(Map.of(
            String.class, new StringSerializer(),
            byte[].class, new ByteArraySerializer()));
        recoveryFactory = new DefaultKafkaProducerFactory<Object, Object>(
            properties, keys, values);
        var deadLetterTemplate = new KafkaTemplate<Object, Object>(recoveryFactory);
        var recoverer = new DeadLetterPublishingRecoverer(deadLetterTemplate);
        recoverer.setFailIfSendResultIsError(true);

        var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 2L));
        handler.addNotRetryableExceptions(IllegalArgumentException.class);
        return handler;
    }

    @PreDestroy
    void closeRecoveryProducer() {
        if (recoveryFactory != null) recoveryFactory.destroy();
    }
}
```

The error-handler bean is picked up by Boot's default listener-container configuration. `FixedBackOff(1000,2)` gives two retry attempts, separated by one second, after the first failure for retryable listener errors. Contract violations are classified nonretryable. Default fatal classifications also cover certain deserialization/conversion failures.

After recovery is needed, `DeadLetterPublishingRecoverer` uses the default `.DLT` topic and original partition. `setFailIfSendResultIsError(true)` surfaces failed DLT publication instead of treating it as successful recovery. Only successful processing or recovery should allow progress past the record.

The separate recovery producer sends both ordinary event objects and raw bytes. That distinction matters: failed JSON deserialization leaves the original raw bytes, which must use `ByteArraySerializer`, not a JSON serializer. Keys likewise support ordinary strings or failed raw bytes. The recovery template is local to this bean so it does not replace Boot's normal application producer template. `@PreDestroy` closes the custom factory's producer resources when the context shuts down. Error handlers, Raw-byte recovery requirement.

All three demo groups share one DLT topic; recovery adds metadata such as original topic/partition/offset and exception details. Production teams may prefer separate DLTs per subscriber. A DLT needs monitoring, ownership, and controlled replay. DLT is a quarantine workflow, not a guarantee that the business action eventually succeeds.

I. Send and inspect

Run `mvn spring-boot:run`, then:

```
$orderBody = @{orderId='O-100'; amount=499.00; currency='INR'} | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri 'http://localhost:8083/demo/order-events' -ContentType 'application/json'
    -Body $orderBody
```

Expected: a generated event with version 1 and HTTP 202; each of the three listener groups logs the same event ID. Consumers may print in any order. Restarting a group with its committed offsets should resume, not reprocess its entire history. Changing group IDs creates new logical subscriptions and can replay retained data under `earliest`.

To demonstrate a duplicate deliberately, send the **same JSON event with the same eventId** directly to the topic using a Kafka producer tool. Calling the HTTP endpoint twice creates two distinct event IDs and therefore demonstrates repeated requests, not redelivery of one event.

5. Real-World Failure Handling

Same event twice: durable idempotency

An in-memory `Set<UUID>` loses its contents on restart and is not sufficient across instances. Use a database uniqueness constraint on `(consumerName,eventId)`. In one local database transaction:

```
BEGIN
  Insert processed-event marker for (consumerName, eventId).
  If this unique marker already exists: finish without repeating the effect.
  Apply the local business update.
COMMIT
Return successfully; then let the container record consumption progress.
```

The marker and business update must commit together. Marking processed before a separately committed update can lose work. Checking existence and then inserting without a unique constraint is vulnerable to races. When a uniqueness conflict occurs, handle it using the database's atomic insert/conflict semantics or a fresh transaction; do not blindly continue in a transaction already marked rollback-only.

For an external payment API, pass a stable business idempotency key as well. A database transaction cannot roll back an already completed external charge. Event-ID deduplication prevents redelivery effects; business-key constraints also prevent two different event IDs from creating the same charge.

Consumer crashes

- Crash before the effect: redelivery should execute it.
- Crash after database commit but before offset commit: redelivery can occur; idempotency prevents repeating the effect.
- Commit offset before effect: crash can permanently skip work.

Consumer-group rebalancing reallocates partitions after membership changes. Long processing may exceed poll limits and trigger reassignment; tune processing/poll settings and bound external calls instead of raising limits without understanding capacity.

Late or out-of-order events

Distinguish `occurredAt` from processing time. Do not overwrite a newer state with an older fact just because it arrived later. Add per-entity sequence/version when the business needs it, and apply allowed state transitions conditionally. A timestamp alone is not a reliable total order across machines. For missing earlier versions, choose buffering, reconciliation, or rejection according to requirements. Retry topics and DLT replay can also change observable processing order.

Broker unavailable

Producer buffering/retries are bounded. Observe failed futures, capacity, and deadlines. Consumers cannot read until connectivity returns; retained events can be processed after recovery. Monitor publication failures and consumer lag. Do not claim that the broker makes services completely independent of infrastructure availability.

Database updated but publication fails: Outbox Pattern

Two independent operations, database commit and broker send, create a dual-write gap. Reversing them merely creates the opposite gap: an event may be published for a rolled-back order.

```
Order transaction: write order + write outbox event row --> one DB COMMIT
                                                            |
                                                            v
Publisher reads pending outbox rows --> Kafka --> mark sent
```

The **outbox** stores the event in the same local transaction as the order. A publisher later sends it with retries. A crash after send but before “sent” marking can resend it, so outbox does not remove the need for idempotent consumers. Preserve the row's event ID across retries and plan per-order publication order. This solves the relevant publication gap without claiming an arbitrary DB/Kafka/global transaction.

Schema and versioning

Use stable field meanings. Adding an optional field is often compatible when readers ignore unknown fields and have defaults; it is not automatically safe for every consumer. Removing/renaming required fields, changing types, or changing amount/currency meaning is breaking. Introduce a versioned contract/topic when necessary, deploy compatible consumers, migrate producers, and retire old contracts only after replay/retention requirements are addressed. Java class names are implementation details, not a portable event schema.

Important Interview Questions

10 beginner questions

1. **What is EDA?** An architecture where components publish facts and other components react through event channels.
2. **What is an event?** A record of something that happened, such as OrderCreated, distinct from a command to perform an action.
3. **What does a producer do?** Constructs and publishes records to a broker/channel.
4. **What does a consumer do?** Reads records and executes its own reaction.
5. **What is a broker?** An intermediary that accepts, stores, and makes messages available to consumers.
6. **Why asynchronous communication?** The producer need not wait for every subscriber's business processing.
7. **What is eventual consistency?** Related states become consistent after independent processing, rather than at one instant.
8. **What is an event schema?** The agreed fields, types, and meanings needed to interpret an event.
9. **What is a DLQ?** A separate destination for failed records that need investigation or later recovery.
10. **Is Kafka the definition of EDA?** No. Kafka is one implementation technology for event transport/storage.

10 intermediate questions

1. **Why do duplicates happen?** Retries and crashes around effect/offset boundaries can repeat publication or delivery.
2. **How do you make consumption idempotent?** Use durable unique event markers with atomic local effects, plus business/external idempotency keys where needed.
3. **Does Kafka guarantee global topic order?** No. Order is within each partition.
4. **What does a consumer group do?** Shares partition work among instances of one logical subscriber.
5. **How do three services all receive an event?** Give each independent service its own consumer group.

6. **Broker ACK vs consumer commit?** One confirms broker acceptance under configured durability; the other records a group's reading progress.
7. **Does producer idempotence prevent duplicate charges?** No. It is not business deduplication or external-effect protection.
8. **Notification vs event-carried state?** Notification carries an identifier; carried state includes data consumers need without fetching it immediately.
9. **How do you evolve a schema?** Prefer compatible additions, preserve meanings, and version breaking contracts with a migration plan.
10. **Why an outbox?** To atomically record the business update and intent to publish in one local database transaction.

5 scenario-based questions

1. **Payment receives the same event twice. What do you do?** Deduplicate durably, enforce one payment per business key, and reuse the external provider's idempotency key.
2. **A consumer crashes after updating its DB. What happens?** If its offset was not committed, the event can be redelivered; atomic deduplication makes that safe.
3. **Cancelled arrives before Created. How do you handle it?** Use entity versions and valid state transitions; buffer/reconcile or reject according to the contract, rather than blindly overwriting state.
4. **Order DB commit succeeds but Kafka send fails. What fixes it?** Write an outbox row with the order in the same DB transaction and publish it later with retries.
5. **Malformed JSON blocks a partition. What is your approach?** Use ErrorHandlingDeserializer, classify the failure, route raw data to a monitored DLT, and avoid infinite poison-record retries.

Common Mistakes

- **All subscribers use one group:** assign distinct group IDs for independent reactions.
- **Commit before the effect:** process successfully before recording consumption progress.
- **In-memory-only deduplication:** use durable atomic constraints and business idempotency.
- **Swallow listener exceptions:** throw failures so retry/recovery policy can act.
- **Assume send() means downstream success:** observe the future and distinguish broker acceptance from business completion.
- **Infinite retries for invalid contracts:** classify permanent errors and send them to a monitored DLT.
- **JSON serializer for failed raw bytes:** recovery producers need byte-array support.
- **Assume timestamps fix ordering:** use partition keys and explicit business versions where required.
- **Publish before committing order state:** use an outbox when durable order/event consistency matters.
- **Replay without planning side effects:** preserve idempotency, review stale events, and monitor replay progress.

Quick Revision

- Events are facts; commands are requests.
- Producer → broker/topic → independent consumer groups.
- Each partition is ordered; the whole topic has no global order guarantee.
- At-least-once processing requires safe repeated effects.
- Broker durability, consumer progress, and business completion are separate boundaries.

- Retry transient failures; quarantine permanent failures.
- Outbox closes the local database/publication gap, but may still deliver duplicates.
- Use compatible schemas and stable event identifiers.

Remember This for Interviews

1. “EDA removes direct subscriber calls, but consumers still depend on the event contract.”
2. “A partition key helps related events stay together; it cannot correct wrong business publication order.”
3. “Different consumer groups each get their own logical view of a topic.”
4. “A successful Kafka send does not mean payment, stock, and notification have completed.”
5. “I expect duplicates and make business effects idempotent.”
6. “Kafka exactly-once guarantees have a defined boundary and do not automatically include external effects.”
7. “The outbox atomically records a business change and the intent to publish it.”
8. “A dead-letter destination needs an operational recovery process.”

Topic 3 - Tailwind CSS

1. What is it?

Tailwind CSS is a utility-first CSS framework. Instead of starting with a custom class such as `.product-card` and writing all its CSS, you combine small classes such as `p-4`, `rounded-lg`, and `bg-white` directly in HTML or JSX.

Each utility has a narrow styling purpose. The build scans source files and generates the CSS required by detected utilities. The browser still uses ordinary CSS; Tailwind is not a replacement for CSS layout knowledge.

It exists to make styling faster and more consistent through reusable spacing, colors, typography, and variants. It reduces repeated naming decisions and makes styles visible alongside markup.

Analogy: assemble a meal from standard ingredients instead of buying one fixed prepared meal. Utilities are ingredients; reusable components are your recipes.

Use it: for custom interfaces, responsive layouts, and teams that want a shared design scale with styling close to components.

Do not force it: a project with an established CSS system may gain little from migration. Complex custom animations or specialist styling may still benefit from normal CSS. Teams that strongly prefer separate stylesheets should weigh readability and maintenance tradeoffs.

Tailwind is normally used in frontend build pipelines. A React interface can call Spring Boot APIs while Tailwind styles the interface. It does not provide API communication, authentication, routing, or backend behavior.

2. Core Concepts

Term	Meaning	Why needed	Example
Utility-first CSS	Compose small single-purpose classes	Express styling without inventing a class for every element	`flex gap-4 p-6`
Utility class	A class mapped to CSS rules	Makes one styling choice	`font-bold`
Design token	Reusable theme value	Keeps visual decisions consistent	Spacing scale, brand color
Variant	Condition under which a utility applies	Handles interaction or environment	`hover:bg-indigo-700`
Breakpoint	Minimum viewport width for responsive utilities	Adapts layout by available space	`md:grid-cols-3`
Mobile-first	Base styles apply first; wider screens override	Keeps small-screen layout explicit	`grid-cols-1 md:grid-cols-3`
Flexbox	Layout mainly along one axis	Aligns bars, rows, and stacks	`flex items-center`
Grid	Layout with rows and columns	Builds structured multi-column layouts	`grid grid-cols-3`
Gap	Spacing between flex/grid children	Avoids per-child margins	`gap-6`

Term	Meaning	Why needed	Example
Arbitrary value	A literal value inside a utility	Supports a special value outside the scale	`max-w-[72rem]`
Source detection	Detect complete class names in source	Determines which CSS is generated	Literal `bg-red-600`
Dark mode	Conditional alternate visual style	Respects theme preferences	`dark:bg-slate-900`

Utilities combine as CSS properties. A `flex` container gives `items-center` and `justify-between` their layout context. `md:` changes the utility's condition, not the element's semantic meaning.

Common utility classes

Values below assume the default theme. Root font size and custom theme tokens can alter computed pixel values.

Category	Classes	Meaning and example
Text color	`text-slate-900`, `text-indigo-600`	Sets text color, e.g. a dark heading
Background	`bg-white`, `bg-indigo-600`, `bg-black/50`	White surface, brand surface, translucent overlay
Font size	`text-sm`, `text-base`, `text-xl`, `text-4xl`	Small copy through large heading; `text-base` is normally 1rem
Weight	`font-normal`, `font-medium`, `font-semibold`, `font-bold`	Increasing emphasis
Typography	`leading-relaxed`, `tracking-tight`, `text-center`	Line height, letter spacing, alignment
Padding	`p-4`, `px-6`, `py-3`, `pt-2`	All sides, horizontal, vertical, top
Margin	`m-4`, `mx-auto`, `mt-6`, `mb-2`	External spacing; auto horizontal margins center a constrained block
Width	`w-full`, `w-1/2`, `w-12`, `max-w-md`	Parent width, half parent, scale width, maximum width
Height	`h-12`, `min-h-screen`, `h-auto`	Scale height, minimum viewport height, natural height
Border	`border`, `border-2`, `border-slate-300`	Border width and color
Radius	`rounded`, `rounded-lg`, `rounded-full`	Rounded corners through pill/circle
Shadow	`shadow-sm`, `shadow-md`, `shadow-lg`	Increasing surface elevation cues
Flex direction	`flex`, `flex-col`, `flex-row`	Enable flex, stack, or row
Flex alignment	`items-center`, `justify-between`, `justify-center`	Cross-axis centering; main-axis distribution or centering
Flex sizing	`flex-1`, `shrink-0`, `flex-wrap`	Flexible child, prevent shrink, allow wrap

Category	Classes	Meaning and example
Grid	`grid`, `grid-cols-1`, `grid-cols-3`, `col-span-2`	Enable grid, choose columns, span two columns
Gap	`gap-4`, `gap-x-6`, `gap-y-2`	Both axes, horizontal, vertical
Position	`relative`, `absolute`, `fixed`, `sticky`, `top-0`	Positioning mode and offsets
Display	`block`, `inline-block`, `hidden`, `md:flex`	Show/hide/change formatting context
Overflow	`overflow-hidden`, `overflow-auto`	Clip or scroll overflowing content
Hover	`hover:bg-indigo-700`	Change background when hovered
Focus	`focus:border-indigo-600`, `focus-visible:outline-2`	Styles for focused controls; visible keyboard focus
Disabled	`disabled:opacity-50`, `disabled:cursor-not-allowed`	Visual state when the actual control is disabled
Transition	`transition-colors`, `duration-150`	Smooth color changes over a chosen duration
Dark theme	`dark:bg-slate-900`, `dark:text-white`	Alternate background/text under dark conditions

`p-4` is normally `4 × 0.25rem = 1rem`, usually 16px with a 16px root font. `p-8` is normally 2rem, not 8px. `px` means horizontal padding, not pixels. `text-*` can express either color or font size depending on the suffix.

Breakpoints

Prefix	Default minimum width	Example
No prefix	Applies at all widths unless overridden	`grid-cols-1`
`sm:`	40rem, normally 640px	`sm:px-6`
`md:`	48rem, normally 768px	`md:grid-cols-2`
`lg:`	64rem, normally 1024px	`lg:grid-cols-3`
`xl:`	80rem, normally 1280px	`xl:max-w-6xl`
`2xl:`	96rem, normally 1536px	`2xl:px-12`

`sm:` means **640px and wider**, not “only phones.” `grid-cols-1 md:grid-cols-2 lg:grid-cols-3` means one column initially, two at `md`, and three at `lg`. Breakpoints are customizable. Include `<meta name="viewport" content="width=device-width, initial-scale=1.0">` so mobile browsers use the intended viewport. Responsive design.

3. Architecture / Working Flow

```
HTML / React source with literal utility classes
                    |
                    v
Tailwind build integration scans source tokens
                    |
                    v
Generate utility rules and conditional variants
                    |
                    v
Compiled CSS included in the page
                    |
                    v
Browser lays out elements and applies responsive/state rules
```

1. You write complete utilities in source.
2. Tailwind detects candidates in files covered by source detection.
3. The compiler generates corresponding CSS, plus the imported theme/base layers.
4. Vite bundles the imported stylesheet with the frontend.
5. The browser evaluates normal CSS, media queries, selectors, and layout rules.

Tailwind does not execute JavaScript expressions to discover class names. `bg-${color}-600` is therefore unreliable; the complete class may never appear in scanned source. Prefer maps of literal complete classes. In v4, `@source` can include sources not detected automatically. Source detection.

The order of conflicting classes in an HTML string is not a safe override strategy. `p-2 p-6` does not mean “the last class wins”; generated CSS order and specificity determine the result. Choose one base utility for each property and condition.

4. Practical Implementation

A. Modern Tailwind 4 setup with React/Vite

In an existing React/Vite project:

```
npm install tailwindcss @tailwindcss/vite
```

`vite.config.js`:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`src/index.css`:

```
@import "tailwindcss";

@theme {
  --color-brand-600: #4f46e5;
  --color-brand-700: #4338ca;
}
```

Import `./index.css` from the React entry file. The Vite plugin compiles utilities; `@import` brings in Tailwind; `@theme` defines tokens that create utilities such as `bg-brand-600`. No JavaScript `tailwind.config.js` is required for this basic v4 setup. These commands assume React/Vite already exists; Tailwind does not create the React project itself. Tailwind with Vite.

Older tutorials: Tailwind v3 commonly uses `@tailwind base/components/utilities` and a `content` array in `tailwind.config.js`. Those are version-specific workflows. Tailwind 4 uses CSS-first configuration and modern integrations. Do not mix v3 installation commands into this v4 example. The browser/CDN playground approach is useful for demos, but use a build integration for production.

B. Break down a utility combination

```
<div class="flex items-center justify-between p-4 bg-white shadow-md rounded-lg">
  <span>Order status</span>
  <span>Accepted</span>
</div>
```

- `flex`: creates a flex container, row direction by default.
- `items-center`: centers children on the cross axis; here, vertically.
- `justify-between`: places children toward opposite ends along the main axis.
- `p-4`: internal padding of 1rem on all sides under the default spacing token.
- `bg-white`: white background.
- `shadow-md`: medium box shadow.
- `rounded-lg`: large predefined corner radius.

This creates a spaced status row. If changed to `flex-col`, the main and cross axes change, so the interpretation of the alignment utilities changes too.

C. Button

```
<button type="button"
  class="rounded-lg bg-indigo-600 px-5 py-3 font-semibold text-white
         transition-colors hover:bg-indigo-700
         focus-visible:outline-2 focus-visible:outline-offset-2
         focus-visible:outline-indigo-600 disabled:cursor-not-allowed disabled:opacity-50">
  View order
</button>
```

Padding creates a usable hit area; background/text classes set contrast; hover gives pointer feedback; focus-visible retains keyboard feedback; disabled variants apply only if the button actually has the `disabled` attribute. `type="button"` prevents accidental form submission. Styling alone does not implement its action. Do not remove focus styles without a visible replacement.

D. Navbar

```
<header class="border-b border-slate-200 bg-white">
  <nav aria-label="Main navigation"
    class="mx-auto flex max-w-6xl flex-wrap items-center justify-between gap-4 px-4 py-4 sm:px-6">
    <a href="/" class="text-xl font-bold text-slate-900">ShopNotes</a>
    <div class="flex flex-wrap items-center gap-4 text-sm font-medium text-slate-700">
      <a href="#products" class="hover:text-indigo-600 focus-visible:outline-2">Products</a>
      <a href="#login" class="hover:text-indigo-600 focus-visible:outline-2">Sign in</a>
    </div>
  </nav>
</header>
```

`max-w-6xl mx-auto` caps and centers the content. `justify-between` separates brand and links; `flex-wrap` permits narrow-screen wrapping; gap keeps items apart; `sm:px-6` increases horizontal padding on wider viewports. `nav` and `aria-label` express navigation semantics. A hidden menu would require an accessible toggle; this example keeps the links visible and wrap-capable.

E. Card in React

`ProductCard.jsx`:

```
export default function ProductCard({ product }) {
  return (
    <article className="flex h-full flex-col rounded-xl border border-slate-200 bg-white p-6 shadow-sm">
      <h3 className="text-lg font-semibold text-slate-900">{product.name}</h3>
      <p className="mt-2 flex-1 leading-relaxed text-slate-600">{product.description}</p>
      <p className="mt-4 text-xl font-bold text-indigo-600">{product.priceLabel}</p>
      <button type="button"
        className="mt-4 w-full rounded-lg bg-indigo-600 px-4 py-2 font-medium text-white
                   hover:bg-indigo-700 focus-visible:outline-2 focus-visible:outline-offset-2">
        View details
      </button>
    </article>
  );
}
```

React uses `className`; utilities are otherwise the same. `flex-col` makes a vertical card and `flex-1` lets the description occupy remaining space, aligning bottom content in equal-height cards. The supplied object needs `name`, `description`, and `priceLabel`. The component styles a product; its details action must be connected by the application. Avoid treating a styled button as completed interaction logic.

F. Login form

```
<section id="login" class="mx-auto max-w-md px-4 py-12">
  <form method="post" action="/login" class="space-y-5 rounded-2xl border border-slate-200 bg-white p-6
      shadow-md">
    <h2 class="text-2xl font-bold text-slate-900">Sign in</h2>
    <div>
      <label for="email" class="mb-2 block text-sm font-medium text-slate-700">Email</label>
      <input id="email" name="email" type="email" autocomplete="username" required
        class="w-full rounded-lg border border-slate-300 px-3 py-2
               focus:border-indigo-600 focus:outline-2 focus:outline-indigo-600">
    </div>
    <div>
      <label for="password" class="mb-2 block text-sm font-medium text-slate-700">Password</label>
      <input id="password" name="password" type="password" autocomplete="current-password" required
        class="w-full rounded-lg border border-slate-300 px-3 py-2
               focus:border-indigo-600 focus:outline-2 focus:outline-indigo-600">
    </div>
    <button type="submit"
      class="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white
             hover:bg-indigo-700 focus-visible:outline-2 focus-visible:outline-offset-2">
      Sign in
    </button>
  </form>
</section>
```

This is a **presentation example**, not implemented authentication. `space-y-5` separates direct form children vertically. Labels link to input IDs. Border and focus utilities make field boundaries and keyboard position visible. Email/password types and autocomplete hints aid browser behavior. `method="post"` avoids the default GET submission putting form values in a URL. `/login` is a placeholder action that must be implemented before real use; without a matching backend it returns an error. Use HTTPS and the intended authentication/CSRF handling in the real application. In JSX use `htmlFor`, `className`, and `autoComplete`.

G. Responsive grid

```
import ProductCard from './ProductCard';

const products = [
  { id: 7, name: 'Keyboard', description: 'Compact mechanical keyboard.', priceLabel: '₹2,499' },
  { id: 8, name: 'Mouse', description: 'Lightweight wireless mouse.', priceLabel: '₹999' },
  { id: 9, name: 'Headphones', description: 'Comfortable over-ear headphones.', priceLabel: '₹1,799' },
];

export function ProductGrid() {
  return (
    <section id="products" className="mx-auto max-w-6xl px-4 py-12">
      <h2 className="mb-6 text-2xl font-bold text-slate-900">Products</h2>
      <div className="grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
        {products.map(product => <ProductCard key={product.id} product={product} />)}
      </div>
    </section>
  );
}
```

`grid` enables grid layout; base is one column, `md` gives two, `lg` gives three. `gap-6` separates rows and columns. A stable key identifies each React element; that is the only React prerequisite needed here. Data can later come from Product Service through Gateway; Tailwind only controls appearance.

H. Hero section

```
<section class="bg-slate-950 px-4 py-16 text-white sm:px-6 md:py-24">
  <div class="mx-auto grid max-w-6xl items-center gap-10 md:grid-cols-2">
    <div>
      <p class="text-sm font-semibold uppercase tracking-widest text-indigo-300">ShopNotes</p>
      <h1 class="mt-4 text-4xl font-bold tracking-tight md:text-5xl">Track every order clearly.</h1>
      <p class="mt-6 max-w-xl text-lg leading-relaxed text-slate-300">
        Browse products and follow updates from one simple dashboard.
      </p>
      <a href="#products"
        class="mt-8 inline-block rounded-lg bg-indigo-500 px-6 py-3 font-semibold
               hover:bg-indigo-400 focus-visible:outline-2 focus-visible:outline-offset-4">
        Browse products
      </a>
    </div>
    <div class="rounded-2xl border border-slate-700 bg-slate-900 p-6 shadow-lg">
      <h2 class="text-xl font-semibold">Order O-100</h2>
      <p class="mt-3 text-slate-300">Status: Accepted</p>
      <p class="mt-2 text-slate-300">Updates appear as your order progresses.</p>
    </div>
  </div>
</section>
```

The hero is one column by default and two at `md`. Larger type and vertical padding apply on wider screens. `max-w-xl` prevents excessively long text lines. The right card is static sample content, not live order state. An anchor is appropriate because the action navigates to another location.

I. Dark mode and literal class maps

By default in v4, `dark:` follows the system `prefers-color-scheme`. To use a manual `.dark` ancestor instead, add this to the stylesheet:

```
@custom-variant dark (&:where(.dark, .dark *));
```

Then use paired colors:

```
<div class="rounded-lg bg-white p-6 text-slate-900 dark:bg-slate-900 dark:text-slate-100">
  Readable in either theme.
</div>
```

With the custom variant, adding `class="dark"` to `<html>` activates those rules. Change text, borders, and surfaces together; swapping only the background often destroys contrast. Persisting and initializing theme choice is application behavior. Dark mode.

For conditional React styling:

```
const statusStyles = {
  accepted: 'bg-blue-100 text-blue-800',
  paid: 'bg-green-100 text-green-800',
  failed: 'bg-red-100 text-red-800',
};

export function StatusBadge({ status }) {
  const style = statusStyles[status] ?? 'bg-slate-100 text-slate-800';
  return <span className={`rounded-full px-3 py-1 text-sm font-medium ${style}`}>{status}</span>;
}
```

Each complete utility appears literally in source, so detection can include it. Avoid generating class fragments like `bg-${color}-100`. Reusable React components are usually clearer than repeated huge markup blocks; use custom CSS when it expresses a complex style better.

Important Interview Questions

10 beginner questions

1. **What is Tailwind CSS?** A utility-first CSS framework that generates styles for small classes composed in markup.
2. **What does utility-first mean?** Build a design from narrow-purpose classes rather than starting with large predefined components.
3. **What does p-4 mean?** Padding on all sides using spacing step four, normally 1rem in the default theme.
4. **Padding vs margin?** Padding is inside an element's boundary; margin is outside it.
5. **What does flex do?** Creates a flex container for one-axis-oriented layout.
6. **What does grid do?** Creates a grid container for row/column layouts.
7. **How do you style hover?** Prefix a utility with `hover:`, such as `hover:bg-indigo-700`.
8. **How do you make rounded corners?** Use radius utilities such as `rounded-lg` or `rounded-full`.
9. **React class or className?** JSX uses `className`.
10. **Does Tailwind provide backend functionality?** No. It generates CSS for presentation.

10 intermediate questions

1. **What is mobile-first styling?** Base styles apply at all widths, and minimum-width variants override them on larger screens.
2. **Does sm: mean only small screens?** No. It applies at the sm minimum width and above unless overridden.
3. **items-center vs justify-center?** They align on the cross axis and main axis respectively; axes depend on flex direction.
4. **Why use gap?** It spaces flex/grid children without adding independent margins to each child.
5. **Why can dynamic class strings fail?** Tailwind scans complete source tokens and does not execute interpolation to discover classes.
6. **How do you customize v4's theme?** Use CSS theme tokens with `@theme`, and compatible configuration where needed.
7. **How does dark mode work?** The dark variant follows system preference by default or a custom selector if configured.

8. **Does the last conflicting utility win?** Not necessarily; CSS generation order and specificity decide. Avoid contradictory utilities.
9. **How do you avoid repeated long class lists?** Extract reusable components and shared design patterns; use CSS selectively for complex cases.
10. **Tailwind vs Bootstrap?** Tailwind gives styling primitives for custom composition; Bootstrap also supplies styled components and interaction conventions.

5 scenario-based questions

1. **bg-red-600 works, but bg-${color}-600 does not. Why?** The generated candidate is absent as a complete source token. Map inputs to literal class strings.
2. **Three cards overflow on a phone. How do you fix it?** Start with one column, add wider breakpoint columns, and remove unnecessarily fixed widths/min-widths.
3. **Keyboard users cannot see focus. What do you change?** Add visible focus/focus-visible outline or ring styles and preserve semantic controls.
4. **Dark mode makes text unreadable. Why?** Only the surface changed; pair foreground, border, and surface colors and check contrast in both themes.
5. **A v3 tutorial's setup fails in a v4 project. What do you do?** Use the matching v4 plugin/import/configuration instructions and remove mixed-generation assumptions.

Common Mistakes

- **Treating spacing numbers as pixels:** understand the spacing token and rem units.
- **Assuming breakpoints identify device types:** design by available width and content, not a phone/tablet label.
- **Using alignment without a layout context:** `items-center` needs an applicable flex/grid container.
- **Fixed width everywhere:** use `w-full`, maximum widths, and responsive layout to avoid narrow-screen overflow.
- **Dynamic class fragments:** use complete literal maps or appropriate source inclusion.
- **Removing focus outline:** provide a visible accessible alternative.
- **Styling a div like a button:** use actual buttons/links with the right semantics.
- **Conflicting base utilities:** choose one value per property/condition.
- **Assuming framework guarantees accessibility:** still check labels, contrast, focus, target size, and interaction.
- **Mixing v3 and v4 installation:** follow the installed generation's documentation.

Quick Revision

- Tailwind generates normal CSS from utility candidates.
- Compose layout, spacing, typography, color, and state utilities.
- Base styles first; `sm:`, `md:`, and `lg:` apply from minimum widths upward.
- Flex is useful for rows/stacks; grid is useful for structured columns/rows.
- React uses `className`; utility meanings stay the same.
- Dark mode needs coherent foreground/background choices.
- Source detection requires complete class names.
- Tailwind 4 uses modern CSS-first setup; verify older tutorial assumptions.

Remember This for Interviews

1. “Tailwind changes how I author CSS, not how the browser fundamentally lays out elements.”
2. “Utility-first composition gives flexibility and consistency at the cost of longer markup.”
3. “Responsive variants are mobile-first minimum-width conditions.”
4. “I understand the main/cross axes before choosing Flexbox alignment utilities.”
5. “I keep conditional utilities as complete literal strings for reliable source detection.”
6. “A reusable component can own a consistent set of utilities.”
7. “Accessibility comes from semantics and tested interactions as well as styling.”
8. “A CSS framework styles the interface; it does not implement API or authentication behavior.”

Topic 4 - WebSocket with STOMP

1. What is it?

WebSocket is a protocol for keeping a connection open so client and server can send messages in both directions. **STOMP**, Simple Text Oriented Messaging Protocol, adds messaging commands, headers, destinations, and subscription behavior over a transport such as WebSocket.

With ordinary HTTP request-response, a browser requests something and the server returns a response. That is excellent for fetching products or creating an order. A chat app also needs incoming messages whenever other users send them. Repeatedly asking “anything new?” can create unnecessary requests and delay updates.

Analogy: HTTP resembles sending a question and receiving a reply. WebSocket resembles an open telephone line. STOMP is an agreed conversation format: which channel you address, which channels you listen to, and what each message means.

Use it: chat, collaborative UI updates, interactive notifications, or applications requiring frequent bidirectional messages with low overhead after connection setup.

Do not default to it: infrequent request-response operations remain simpler with HTTP APIs. One-way server updates may be adequately handled by SSE. Persistent connections add lifecycle, authorization, resource limits, reconnection, and deployment concerns.

WebSocket complements REST. A chat application can use REST to fetch message history and WebSocket/STOMP to deliver new messages. The example focuses on live delivery only.

Required HTTP alternatives

- **Polling:** the browser requests updates periodically. Simple, but wastes calls when nothing changes and latency depends on the interval.
- **Long polling:** the server holds a request until data or timeout, then the client sends another request. Lower idle request frequency, but repeated request lifecycle management remains.
- **Server-Sent Events (SSE):** one long-lived HTTP response sends server-to-client text events. Browser `EventSource` can reconnect; resumption still requires server support and event/history design. Client-to-server actions use separate requests.

Ordinary HTTP is not “incapable of real-time behavior.” These mechanisms work; WebSocket is useful when persistent bidirectional messaging fits the interaction.

2. Core Concepts

Term	Meaning	Why needed	Example
Full-duplex	Both directions can send independently	Supports interactive conversations	Browser sends chat while server sends updates
Persistent connection	A connection remains open between messages	Avoids a new HTTP cycle for each message	One active browser socket
WebSocket handshake	Initial protocol negotiation	Establishes WebSocket transport	Typical HTTP/1.1 Upgrade request → 101
Frame	WebSocket transport unit	Carries data/control information	Text, binary, ping, pong

Term	Meaning	Why needed	Example
STOMP frame	Command + headers + body, with protocol delimiters	Adds message-level meaning	SEND or SUBSCRIBE
Destination	A routing address inside the messaging system	Names where a message goes	`/app/chat.send`, `/topic/messages`
Topic	Convention for a broadcast destination	Distributes live updates to subscribers	Everyone in the room receives a chat
Queue	Convention for a queue-style destination	Supports addressed/work-queue patterns where broker semantics permit	External broker work queue
Subscribe	Register interest in a destination	Enables server delivery to that client	SUBSCRIBE to `/topic/messages`
Send / publish	Submit a message to a destination	Starts application processing or broker delivery	SEND to `/app/chat.send`
Broker	Routes messages to subscribed sessions	Separates publication from subscriber lists	Spring's simple broker
Message mapping	Route inbound application messages to a method	Connects STOMP to Java handlers	`@MessageMapping("/chat.send")`
Session	One connected messaging client context	Holds subscriptions and lifecycle state	One browser tab
Heartbeat	Negotiated periodic activity signal	Helps detect stale connections	STOMP heartbeats
Broker relay	Spring forwards messaging to an external STOMP broker	Shares distribution across app instances	Replace in-process simple broker when scaling

STOMP defines commands such as CONNECT/STOMP, CONNECTED, SEND, SUBSCRIBE, MESSAGE, UNSUBSCRIBE, DISCONNECT, and ERROR. It also defines acknowledgement/receipt/transaction features, but broker implementations may support only a subset. STOMP destination syntax and delivery semantics are broker-specific; a `/queue` prefix alone cannot guarantee durable storage or competing consumption. STOMP 1.2 specification.

Three prefixes that interviews often test

Prefix	Typical Spring purpose	Example flow
`/app`	Routes inbound messages to application handlers	SEND `/app/chat.send` → `@MessageMapping("/chat.send")`
`/topic`	Routes to broker destinations, conventionally broadcasts	SUBSCRIBE `/topic/messages` receives room messages
`/queue`	Routes to broker destinations, conventionally queue-like	Broker-specific behavior; not automatically private
`/user`	Spring's user-destination translation	SUBSCRIBE `/user/queue/errors` gets session/user-targeted output

Spring's **simple broker** maintains subscriptions in memory and dispatches matching messages. It does not turn `/queue/jobs` into a durable competing-consumer queue. If several sessions subscribe to a matching simple-broker queue destination, they can all receive the message. Production work-queue semantics depend on the external broker configuration.

For personal delivery, use Spring user destinations with authenticated identity and destination authorization. `/user/queue/messages` is translated to session-specific destinations. It is different from asking every client to subscribe to a globally shared `/queue/messages`. User destinations.

3. Architecture / Working Flow



1. The browser establishes a transport connection to `/ws`. This is a connection endpoint, not a message destination.
2. On the classic HTTP/1.1 path, Upgrade headers negotiate WebSocket and the server responds 101 Switching Protocols. Some HTTP/2 deployments use extended CONNECT instead; “always 101” is too broad.
3. Once the transport is open, the client negotiates STOMP separately. An open WebSocket is not automatically a completed STOMP connection.
4. The client subscribes to `/topic/messages` after STOMP is connected.
5. SEND to `/app/chat.send` is processed by Spring's application handler after the `/app` prefix is removed for matching.
6. The controller validates content, creates a server-stamped output model, and sends it to the broker destination through `@SendTo`.
7. The broker emits MESSAGE frames to matching currently connected subscriptions. Clients render their bodies.

Neither a transport connection nor a STOMP receipt proves that a user read a chat message. Browser rendering, durable storage, business acknowledgement, and delivery receipts are different concerns.

4. Practical Implementation: Real-Time Chat

A. Complete minimal project structure

```
chat-demo/
  pom.xml
  src/main/java/com/example/chat/
    ChatApplication.java
    WebSocketConfig.java
    ChatInput.java
    ChatMessage.java
    ChatController.java
  src/main/resources/
    application.properties
    static/index.html
```

B. Dependencies

`pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version><relativePath/>
  </parent>
  <groupId>com.example</groupId><artifactId>chat-demo</artifactId><version>1.0.0</version>
  <properties><java.version>21</java.version></properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-websocket</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId>
  </plugin></plugins></build>
</project>
```

The WebSocket starter supplies Spring WebSocket, messaging, and the servlet web runtime. Validation supplies Jakarta validation support for message inputs. This is a separate MVC application, not the WebFlux Gateway project.

`ChatApplication.java`:

```
package com.example.chat;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ChatApplication {
    public static void main(String[] args) {
        SpringApplication.run(ChatApplication.class, args);
    }
}
```

The application scans its package and starts the server. Put `server.port=8084` in `application.properties`.

C. WebSocket configuration

`WebSocketConfig.java`:

```
package com.example.chat;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOrigins("http://localhost:8084");
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.setApplicationDestinationPrefixes("/app");
        registry.enableSimpleBroker("/topic", "/queue");
    }
}
```

Annotations and methods:

- `@Configuration`: registers configuration with Spring.
- `@EnableWebSocketMessageBroker`: enables STOMP-over-WebSocket infrastructure, channels, handler mappings, and broker integration.
- `WebSocketMessageBrokerConfigurer`: callbacks for endpoint, broker, channel, and transport configuration.
- `registerStompEndpoints`: chooses where a client establishes the transport. `/ws` must match the client's connection URL.
- `setAllowedOrigins`: permits the precise browser origin used by this demo. Origins include scheme, host, and port. Add a separately hosted frontend's exact origin if needed.
- `configureMessageBroker`: configures application and broker destination routing.
- `setApplicationDestinationPrefixes`: sends `/app/...` messages to application handlers.
- `enableSimpleBroker`: activates an in-memory broker for the given destination prefixes.

This uses native WebSocket; SockJS is not required. SockJS is an optional compatibility fallback with different HTTP endpoint/client setup, not another name for STOMP. Do not use a SockJS HTTP URL as a native WebSocket URL. Enable STOMP in Spring.

D. Models and controller

`ChatInput.java`:

```
package com.example.chat;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record ChatInput(
    @NotBlank @Size(max = 40) String sender,
    @NotBlank @Size(max = 500) String content
) {}
```

`ChatMessage.java`:

```
package com.example.chat;

import java.time.Instant;

public record ChatMessage(String sender, String content, Instant sentAt) {}
```

`ChatController.java`:

```
package com.example.chat;

import jakarta.validation.Valid;
import java.time.Instant;
import org.springframework.messaging.handler.annotation.MessageExceptionHandler;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.SendTo;
import org.springframework.messaging.simp.annotation.SendToUser;
import org.springframework.stereotype.Controller;

@Controller
public class ChatController {
    @MessageMapping("/chat.send")
    @SendTo("/topic/messages")
    public ChatMessage send(@Valid ChatInput input) {
        return new ChatMessage(input.sender().trim(), input.content().trim(), Instant.now());
    }

    @MessageExceptionHandler(Exception.class)
    @SendToUser(value = "/queue/errors", broadcast = false)
    public String invalidMessage(Exception exception) {
        return "Message rejected. Check the sender and content limits.";
    }
}
```

`@Controller` registers messaging handlers; this method is not an HTTP REST endpoint. `@MessageMapping` matches inbound application destinations after prefix removal. `@SendTo` publishes the returned object to the topic. Spring message converters map incoming JSON to `ChatInput` and output records to JSON. `@Valid` enforces the model constraints; the server generates the timestamp rather than trusting a client clock.

`@MessageExceptionHandler` handles errors in this controller's message handling; it is not an HTTP `@ExceptionHandler`. `@SendToUser(...,broadcast=false)` targets the sending session's error subscription rather than broadcasting private errors. It can reply to an unauthenticated sending session for this demo; this does not turn the sender field into trusted identity. Annotated message handlers.

This is a public local chat demo. `sender` is a display name that can be forged. In an authenticated application, derive identity from `Principal`, authorize SEND and SUBSCRIBE destinations, and never trust a client-provided identity. Validation limits lengths but does not itself provide transport-size limits, spam prevention, or room permissions.

E. Complete browser client

`src/main/resources/static/index.html`:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>STOMP chat demo</title>
</head>
<body>
  <h1>Chat demo</h1>
  <p id="status" role="status">Connecting…</p>
  <form id="chat-form">
    <label for="sender">Display name</label>
    <input id="sender" required maxlength="40" value="Ravi">
    <label for="content">Message</label>
    <input id="content" required maxlength="500">
```

```
    <button id="send" type="submit" disabled>Send</button>
  </form>
  <ul id="messages" aria-live="polite"></ul>

  <script src="https://cdn.jsdelivr.net/npm/@stomp/stompjs@7.2.0/bundles/stomp.umd.min.js"></script>
  <script>
    const status = document.getElementById('status');
    const send = document.getElementById('send');
    const messages = document.getElementById('messages');
    const client = new StompJs.Client({
      brokerURL: `${location.protocol === 'https:' ? 'wss' : 'ws'}://${location.host}/ws`,
      reconnectDelay: 3000,
      connectionTimeout: 10000,
    });

    client.onConnect = () => {
      client.subscribe('/topic/messages', frame => {
        try {
          const message = JSON.parse(frame.body);
          const item = document.createElement('li');
          item.textContent = `${message.sender}: ${message.content} (${message.sentAt})`;
          messages.appendChild(item);
        } catch {
          status.textContent = 'Received an invalid message.';
        }
      });
      client.subscribe('/user/queue/errors', frame => {
        status.textContent = frame.body;
      });
      status.textContent = 'Connected';
      send.disabled = false;
    };

    client.onWebSocketClose = () => {
      status.textContent = 'Disconnected; trying to reconnect…';
      send.disabled = true;
    };
    client.onWebSocketError = () => {
      status.textContent = 'Connection error.';
      send.disabled = true;
    };
    client.onStompError = () => {
      status.textContent = 'Messaging server reported an error.';
      send.disabled = true;
    };

    document.getElementById('chat-form').addEventListener('submit', event => {
      event.preventDefault();
      if (!client.connected) return;
      const sender = document.getElementById('sender').value.trim();
      const content = document.getElementById('content').value.trim();
      if (!sender || !content) return;
      client.publish({
        destination: '/app/chat.send',
        headers: {'content-type': 'application/json'},
        body: JSON.stringify({sender, content}),
      });
      document.getElementById('content').value = '';
    });

    client.activate();
    window.addEventListener('pagehide', () => { void client.deactivate(); });
  </script>
</body>
</html>
```

The UMD script exposes `StompJs.Client` and requires internet access for this demo's library load. A production frontend should bundle the dependency through its package/build system and reviewed asset policy.

`brokerURL` uses the page's actual host and selects encrypted `wss` on HTTPS pages. `activate()` starts the connection. `onConnect` runs after each successful STOMP connection, so subscriptions are recreated after reconnect. `publish()` requires an active connection and sends serialized JSON to the application destination. Subscription callbacks receive broker messages and parse their bodies.

`textContent` renders untrusted chat text without interpreting HTML, preventing this display path from injecting markup. Do not substitute `innerHTML` with message content. The button is disabled when disconnected. Error callbacks show sanitized statuses. `deactivate()` stops reconnect attempts and disconnects; browser page teardown is best-effort, so server-side lifecycle handling is still required. StompJS client lifecycle and API.

F. Run and verify manually

Run `mvn spring-boot:run`. Open `http://localhost:8084/` in two tabs. Use different display names, wait for Connected, and send a message. Both tabs should display the same server-stamped message.

Close one tab: its subscriptions disappear. Reopen: it receives future messages after subscribing, **not past messages**. Restart the server: the client attempts reconnection and resubscribes, but messages sent during the gap are not recovered by this simple demo.

Use browser Network tools to inspect the `/ws` handshake and STOMP frames. A successful handshake followed by no messages suggests STOMP or destination/subscription problems. A failed handshake suggests URL, origin, proxy upgrade support, or HTTP authentication problems.

5. Reliability, Security, and Scaling

- **Disconnects:** detect them, reconnect with backoff, and recreate subscriptions. A reconnect is a new session; old subscriptions are not retained automatically.
- **Heartbeats:** WebSocket ping/pong and STOMP heartbeats are different mechanisms. Configure compatible client/server heartbeat support. For Spring's simple broker, enable its heartbeat settings with an appropriate task scheduler rather than assuming client settings alone enable server heartbeats.
- **History and recovery:** persist messages and expose a history/cursor mechanism if missed messages matter. The simple broker is not a durable event log.
- **Multiple Spring instances:** each simple broker knows only its own sessions. A send handled by one instance does not automatically reach clients attached to another. An external STOMP broker relay can provide shared distribution; design user-destination behavior and reconnect routing accordingly.
- **Authentication:** Spring normally associates the HTTP handshake's authenticated principal with the WebSocket session. Browser native WebSocket does not allow arbitrary HTTP Authorization headers. A STOMP CONNECT Authorization header is a STOMP header, not a handshake header, and needs explicit inbound interception/validation if using that strategy. Spring does not automatically authenticate arbitrary STOMP `login`/`passcode` headers for this setup. Spring authentication model.
- **Authorization:** restrict SEND to intended `/app` paths, restrict SUBSCRIBE by user/room, and normally reject client SEND to server broadcast destinations. Protecting only `/ws` does not authorize every later message.
- **Origin and CSRF:** origin restrictions reduce browser cross-origin abuse but do not authenticate nonbrowser clients. WebSocket HTTP handshakes and STOMP CONNECT security need the correct Spring Security configuration. Cookie/session applications require attention to CSRF protections.
- **Limits:** bound message/frame sizes, outbound buffers, send time, per-user connection/message rate, and slow clients. A persistent connection consumes resources even when idle.
- **Transport deployment:** use WSS externally and configure proxy upgrade/idle timeouts. Keep tokens out of URLs and logs. Expiry during a long session needs a defined reauthentication/disconnect policy.

Important Interview Questions

10 beginner questions

1. **What is WebSocket?** A persistent bidirectional transport for exchanging messages after connection establishment.
2. **What is full-duplex?** Client and server can send independently in both directions.
3. **Why not poll for everything?** Polling adds idle requests and interval-dependent latency; WebSocket suits frequent bidirectional updates.
4. **What is STOMP?** A text-oriented messaging protocol adding commands, headers, and destinations over a transport.
5. **Why STOMP over WebSocket?** It supplies an established message/subscription model instead of requiring a custom protocol.
6. **What is a destination?** A messaging address used for application routing or broker subscriptions.
7. **What does Subscribe do?** Registers a session's interest in messages at a destination.
8. **What does @MessageMapping do?** Routes inbound application messages to a Java method.
9. **What does @SendTo do?** Sends a handler's return value to a configured broker destination.
10. **What is the simple broker?** Spring's in-memory subscription-based message dispatcher.

10 intermediate questions

1. **/ws vs /app/chat.send?** `/ws` opens the transport; `/app/chat.send` is a message destination after connection.
2. **/app vs /topic?** Application-handler routing versus broker destination routing.
3. **Does /queue automatically mean private or durable?** No. Its semantics depend on the broker; use authorized user destinations for personal delivery.
4. **WebSocket vs STOMP?** WebSocket transports messages; STOMP defines a message-level protocol that can use that transport.
5. **WebSocket vs SSE?** WebSocket is bidirectional; SSE streams server-to-client events over HTTP, with separate client requests.
6. **Are subscriptions restored automatically after reconnect?** Not on the server; clients must subscribe again, usually in onConnect.
7. **Can simple broker recover missed messages?** No durable history is provided; add persistence and a recovery API/cursor if needed.
8. **How do you send to one user?** Use Spring user destinations with authenticated identity and authorized subscription handling.
9. **How is WebSocket identity established?** Commonly from the authenticated HTTP handshake; token-based STOMP authentication requires explicit handling.
10. **How do multiple app instances broadcast consistently?** Use shared broker/distribution infrastructure rather than isolated simple brokers.

5 scenario-based questions

1. **Handshake succeeds but messages never arrive. What do you check?** STOMP CONNECTED, subscription, `/app` mapping, broker prefixes, payload conversion, and authorization.
2. **Chat stops after server restart. How do you fix it?** Reconnect and subscribe again in onConnect; recover missed history separately if required.

3. **A user subscribes to another user's messages. What prevents it?** Enforce inbound destination authorization and use verified user destinations instead of client-supplied identity paths.
4. **Two app instances each show only some clients' messages. Why?** Their simple brokers are isolated; add an external broker relay or equivalent shared distribution.
5. **A chat payload contains a script tag. What do you do?** Render it as text, validate limits, and enforce safe content handling; never inject raw HTML into the page.

Common Mistakes

- **Use endpoint URL as message destination:** connect to `/ws`, publish to `/app/...`, subscribe to `/topic/...`.
- **Publish before STOMP connects:** wait for `onConnect` and check active connection.
- **Subscribe only once outside reconnect logic:** recreate subscriptions after each connection.
- **Trust sender in JSON:** derive authenticated identity from the session principal.
- **Assume `/queue` is private:** use Spring user destinations plus authorization.
- **Treat simple broker as durable:** store history explicitly when recovery matters.
- **Mix SockJS and native WebSocket URLs:** use a compatible transport/client setup.
- **Leave connections active on component unmount:** deactivate and unsubscribe during application cleanup.
- **Use ws on an HTTPS page:** use wss and correct proxy configuration.
- **Only secure the handshake:** authorize SEND and SUBSCRIBE destinations too.
- **Render chat with innerHTML:** use safe text rendering or a reviewed sanitizer when HTML is a deliberate feature.

Quick Revision

- WebSocket transport and STOMP messaging are separate layers.
- `/ws` connects; `/app` invokes handlers; `/topic` uses broker destinations.
- `@MessageMapping` handles incoming messages; `@SendTo` publishes returned output.
- Simple broker stores subscriptions in memory, not durable message history.
- `/queue` naming alone does not define competing-consumer or private semantics.
- Reconnect and resubscribe; design missed-message recovery separately.
- Authorize message destinations and derive identity from verified session context.

Remember This for Interviews

1. “WebSocket gives a bidirectional transport; STOMP gives messaging conventions on top.”
2. “Transport connection success is different from STOMP negotiation and subscription success.”
3. “Application destinations and broker destinations have different roles.”
4. “A queue prefix is not a privacy or durability guarantee.”
5. “A reconnect creates a new session, so subscriptions need to be restored.”
6. “A simple in-process broker does not solve cross-instance message distribution.”
7. “I authorize SEND and SUBSCRIBE, not just the connection endpoint.”
8. “Live delivery and durable message history are separate requirements.”

Topic 5 - JUnit

1. What is it?

Testing compares actual software behavior with expected behavior. A test provides inputs, executes code, and checks an outcome. Automated tests make these checks repeatable.

JUnit is a Java testing framework. It identifies tests, runs lifecycle methods, evaluates assertions, integrates with build/IDE tools, and reports failures. It does not decide what your application should do; you define the expected behavior.

Unit testing checks a small unit, usually a class or method, with controlled dependencies. **Integration testing** checks cooperating components and infrastructure at an explicitly chosen boundary. The boundary matters more than a single annotation.

Analogy: testing an individual switch is a unit check; checking the assembled lighting circuit is an integration check.

Automated tests give fast regression feedback, document expected behavior, and make refactoring safer. A passing suite only covers the cases and boundaries it actually checks. It does not prove all requirements, security, concurrency, or infrastructure are correct.

Use unit tests: for business decisions, validation, calculations, and error behavior that should run quickly and deterministically.

Use integration tests: when wiring, configuration, HTTP conversion, repository queries, or external interactions need verification.

Do not use one huge full-context test for every method: it is slow and makes failure causes unclear. Do not test framework mechanics or trivial getters without a meaningful risk. Choose tests based on behavior and failure modes.

This chapter teaches **JUnit 5's Jupiter API**, as requested. JUnit 5 historically comprises Platform (launching/engines), Jupiter (programming model/engine), and Vintage (legacy JUnit 3/4 support). Modern JUnit 6 continues Jupiter with a newer baseline; JUnit 5 is not the latest major release. For these examples, import `org.junit.jupiter.api.Test`, not JUnit 4's `org.junit.Test`.

2. Core Concepts

Term	Meaning	Why needed	Example
Test case	A particular behavior under conditions	Makes requirements executable	Missing user throws an exception
Assertion	Check expected vs actual outcome	Detects incorrect behavior	`assertEquals(5, result)`
Fixture	Data/objects prepared for a test	Establishes repeatable conditions	A known repository response
Lifecycle	Setup → test → cleanup	Keeps cases isolated	`@BeforeEach` resets calculator
Arrange-Act-Assert	Prepare → execute → check	Keeps intent readable	Stub repository → call service → assert user
Mock	Controlled replacement for a dependency	Isolates the unit and observes interactions	Fake UserRepository via Mockito

Term	Meaning	Why needed	Example
Stubbing	Specify a mock's response to a call	Controls branch inputs	Return Optional.empty for missing ID
Verification	Check an interaction occurred	Tests important collaboration	Verify deleteById was invoked
Parameterized test	Run one behavior over multiple input values	Covers a family of cases concisely	Several even integers
Test slice	A selected subset of Spring configuration	Checks one application layer efficiently	`@WebMvcTest`
Application context	Spring's configured bean graph	Needed to verify real wiring	`@SpringBootTest`
MockMvc	Exercise MVC request dispatch without a listening server	Checks HTTP mappings, JSON, validation, responses	Perform GET and expect 200

JUnit runs and asserts; Mockito supplies controlled dependencies; Spring Test supplies context and MVC testing support. They cooperate but solve different problems.

Lifecycle annotations

Annotation	Purpose	Example / caution
`@Test`	Marks a test method	No main method needed
`@BeforeEach`	Runs before each test invocation	Fresh fixture every case
`@AfterEach`	Runs after each invocation	Close resources created for that case
`@BeforeAll`	Runs once before the class's tests	Usually static under default lifecycle
`@AfterAll`	Runs once after the class's tests	Usually static under default lifecycle
`@DisplayName`	Human-readable test/report label	“Missing user returns 404”
`@Disabled`	Explicitly skips a test/class	Add a reason and a plan; not a failure fix
`@ParameterizedTest`	Repeats a test with supplied arguments	Runs once per input
`@ValueSource`	Supplies simple literal inputs	`ints={0,2,4}`

The default Jupiter lifecycle creates a fresh test instance for each test method. `@TestInstance(PER_CLASS)` changes this and permits nonstatic all-class methods, but shared mutable state then needs extra discipline. Tests should not rely on execution order. Setup/cleanup also applies to parameterized invocations. Jupiter annotations and lifecycle.

Assertions

Assertion	Check	Example
`assertEquals`	Expected equals actual	`assertEquals(5, add(2,3))`
`assertNotEquals`	Values differ	`assertNotEquals(6, add(2,3))`

Assertion	Check	Example
`assertTrue`	Condition is true	`assertTrue(isEven(4))`
`assertFalse`	Condition is false	`assertFalse(isEven(3))`
`assertNull`	Reference is null	Missing map lookup
`assertNotNull`	Reference is present	Returned user exists
`assertThrows`	Executable throws the expected type or subtype	Division by zero throws
`assertAll`	Checks several assertions and collects failures	User ID and name together

Use expected first and actual second in `assertEquals` so failure messages read correctly. `assertThrows` returns the exception so you can inspect it; `assertThrowsExactly` exists if exact type rather than subtype matters. `assertAll` does not make the application operations transactional or run them in parallel. For floating-point arithmetic use a justified tolerance; for decimal money prefer the appropriate BigDecimal numeric comparison rather than accidental scale equality. Assertions API.

3. Architecture / Working Flow

```
mvn test / IDE test runner
          |
          v
JUnit Platform discovers test engines and tests
          |
          v
Jupiter executes:
  BeforeAll
     +-- BeforeEach -> test/assertions -> AfterEach
     +-- BeforeEach -> next test       -> AfterEach
  AfterAll
          |
          v
Report: passed / failed / skipped
```

1. Maven's test plugin or the IDE launches the testing platform.
2. Jupiter discovers tests and lifecycle methods.
3. Extensions initialize support such as Mockito mocks or Spring test contexts.
4. A case sets up data, calls code, and checks behavior.
5. Failed assertions or unexpected exceptions fail the test; explicit skips are reported separately.
6. Cleanup releases resources, and the runner aggregates results.

In a Mockito unit test no Boot server/context is necessary. In a WebMvc slice, Spring starts selected MVC infrastructure and a mock service supplies controlled behavior. In a full-context integration test, Spring wires real application beans. None of these automatically verifies a real external database unless it is included in the test boundary.

4. Practical Implementation

A. Dependencies and layout

Create `testing-demo` with the Boot 3.5 parent, Java 21 property, and Maven plugin as previously shown. Use:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

The test starter supplies Jupiter, Mockito integration, Spring Test, and other test support managed by Boot. `scope=test` keeps these out of production runtime dependencies. Standalone Java projects can instead add `org.junit.jupiter:junit-jupiter` and configure a compatible Maven Surefire version; the Boot parent already manages this setup.

Put application types in `src/main/java/com/example/testing` and tests in `src/test/java/com/example/testing`. Create `TestingApplication` in that package using the common Boot entry-point pattern. Test names ending in `Test` are discovered by the standard Maven test lifecycle. `*IT` names require an appropriate integration-test plugin/setup; the integration example here ends in `Test` so `mvn test` runs it.

B. Simple Java before Spring

`Calculator.java`:

```
package com.example.testing;

public class Calculator {
    public int add(int left, int right) { return left + right; }
    public boolean isEven(int value) { return value % 2 == 0; }
    public int divide(int left, int right) {
        if (right == 0) throw new IllegalArgumentException("Divisor must not be zero");
        return left / right;
    }
}
```

Integer division deliberately truncates toward zero; the test must reflect that contract. This small class needs no Spring annotation because calculation itself is ordinary Java.

`CalculatorTest.java`:

```
package com.example.testing;

import java.util.Map;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    private Calculator calculator;

    @BeforeAll
    static void beforeClass() { System.out.println("Before all calculator tests"); }

    @BeforeEach
    void setUp() { calculator = new Calculator(); }

    @AfterEach
```

```
    void tearDown() { calculator = null; }

    @AfterAll
    static void afterClass() { System.out.println("After all calculator tests"); }

    @Test
    @DisplayName("Adds two integers")
    void addsNumbers() {
        int actual = calculator.add(2, 3);
        assertEquals(5, actual);
        assertNotEquals(6, actual);
    }

    @ParameterizedTest
    @ValueSource(ints = {-2, 0, 2, 10})
    void identifiesEvenNumbers(int value) {
        assertTrue(calculator.isEven(value));
    }

    @Test
    void rejectsZeroDivisor() {
        var error = assertThrows(IllegalArgumentException.class,
            () -> calculator.divide(10, 0));
        assertEquals("Divisor must not be zero", error.getMessage());
    }

    @Test
    void groupsIndependentChecks() {
        var names = Map.of(1L, "Ravi");
        assertAll("basic behavior",
            () -> assertEquals(3, calculator.divide(7, 2)),
            () -> assertFalse(calculator.isEven(3)),
            () -> assertNotNull(names.get(1L)),
            () -> assertNull(names.get(99L)));
    }

    @Disabled("Demonstrates a skipped test; remove when studying real test failures")
    @Test
    void skippedExample() { assertEquals(4, calculator.add(2, 2)); }
}
```

`setUp` creates a fresh calculator; resetting it to null only demonstrates cleanup and is unnecessary for this resource-free object. The all-class prints demonstrate lifecycle ordering; real cleanup should close actual resources. Test methods can be package-private. Static assertion imports keep the checks readable.

The parameterized test runs four cases. The lambda in `assertThrows` postpones division until JUnit invokes it; calling division outside that lambda would throw before the assertion. The assertion group checks several independent outcomes. The disabled example is explicitly educational and produces a reported skip.

C. User model, repository, and service

`User.java`:

```
package com.example.testing;

public record User(long id, String name) {}
```

`UserRepository.java`:

```
package com.example.testing;

import java.util.Optional;

public interface UserRepository {
    Optional<User> findById(long id);
    User save(User user);
    boolean existsById(long id);
```

```
    void deleteById(long id);
}
```

`UserNotFoundException.java`:

```
package com.example.testing;

public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(long id) {
        super("User " + id + " was not found");
    }
}
```

`UserService.java`:

```
package com.example.testing;

import org.springframework.stereotype.Service;

@Service
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) { this.repository = repository; }

    public User getUserById(long id) {
        requirePositiveId(id);
        return repository.findById(id).orElseThrow(() -> new UserNotFoundException(id));
    }

    public User createUser(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        return repository.save(new User(0, name.trim()));
    }

    public void deleteUser(long id) {
        requirePositiveId(id);
        if (!repository.existsById(id)) throw new UserNotFoundException(id);
        repository.deleteById(id);
    }

    private static void requirePositiveId(long id) {
        if (id <= 0) throw new IllegalArgumentException("ID must be positive");
    }
}
```

`@Service` makes this class a Spring bean, but its constructor also supports direct testing without Spring. `getUserById` validates ID and converts a missing Optional to the domain exception. `createUser` validates and trims the name before saving; ID 0 means this demo repository assigns an ID. `deleteUser` checks existence before deletion.

This deletion policy chooses “missing → error,” not silent deletion. Its separate existence check is not an atomic database delete guarantee under concurrent changes; a real persistent repository may need a conditional delete/update-count policy. That limitation is part of its implementation contract, not something mocking can verify away.

D. Service unit tests with Mockito

`UserServiceTest.java`:

```
package com.example.testing;

import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
```

```
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock
    UserRepository repository;

    @InjectMocks
    UserService service;

    @Test
    void getsExistingUser() {
        var user = new User(1, "Ravi");
        when(repository.findById(1)).thenReturn(Optional.of(user));
        User actual = service.getUserById(1);
        assertEquals(user, actual);
        verify(repository).findById(1);
    }

    @Test
    void missingUserThrows() {
        when(repository.findById(99)).thenReturn(Optional.empty());
        assertThrows(UserNotFoundException.class, () -> service.getUserById(99));
    }

    @Test
    void createsUserWithTrimmedName() {
        var input = new User(0, "Ravi");
        var saved = new User(10, "Ravi");
        when(repository.save(input)).thenReturn(saved);
        User actual = service.createUser("  Ravi  ");
        assertAll(
            () -> assertEquals(10, actual.id()),
            () -> assertEquals("Ravi", actual.name()));
        verify(repository).save(input);
    }

    @Test
    void blankNameIsRejectedWithoutRepositoryCall() {
        assertThrows(IllegalArgumentException.class, () -> service.createUser("   "));
        verifyNoInteractions(repository);
    }

    @Test
    void deletesExistingUser() {
        when(repository.existsById(1)).thenReturn(true);
        service.deleteUser(1);
        verify(repository).deleteById(1);
    }

    @Test
    void missingUserIsNotDeleted() {
        when(repository.existsById(99)).thenReturn(false);
        assertThrows(UserNotFoundException.class, () -> service.deleteUser(99));
        verify(repository, never()).deleteById(anyLong());
    }
}
```

How the annotations and calls work:

- `@ExtendWith(MockitoExtension.class)` initializes Mockito support for each test; otherwise annotated fields may remain null.
- `@Mock` creates a controlled UserRepository replacement. It does not register a Spring bean.

- `@InjectMocks` creates UserService and supplies matching mocks, using its constructor here. It is Mockito injection, not Spring dependency injection and not a full graph resolver.
- `when(mock.method(...))` selects a mocked invocation to stub.
- `thenReturn(...)` supplies its result for that invocation.
- `verify(mock).method(...)` checks the interaction occurred once by default.
- `never()` checks that an unwanted effect did not occur. `verifyNoInteractions` is appropriate here because invalid input should fail before any repository access.

Arrange a repository response → call the real service → assert result/exception → verify important effect. The repository is mocked; the service is real. Mocking UserService while “testing UserService” would bypass the implementation being tested.

These unit tests cannot prove SQL mappings, database transactions, or Spring wiring. Avoid verifying every incidental method call: over-verification makes refactoring break tests without changing business behavior. Unstubbed mocks return configured/default values; a missing stub can produce misleading nulls or empty results. Strict Mockito stubbing can also flag unused stubs; keep setup specific to the case.

E. Controller and HTTP exception mapping

`CreateUserRequest.java`:

```
package com.example.testing;

import jakarta.validation.constraints.NotBlank;

public record CreateUserRequest(@NotBlank String name) {}
```

`UserController.java`:

```
package com.example.testing;

import jakarta.validation.Valid;
import java.net.URI;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService service;
    public UserController(UserService service) { this.service = service; }

    @GetMapping("/{id}")
    public User get(@PathVariable("id") long id) { return service.getUserById(id); }

    @PostMapping
    public ResponseEntity<User> create(@Valid @RequestBody CreateUserRequest request) {
        User created = service.createUser(request.name());
        return ResponseEntity.created(URI.create("/users/" + created.id())).body(created);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable("id") long id) {
        service.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}
```

UserHttpErrors.java:

```
package com.example.testing;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class UserHttpErrors {
    @ExceptionHandler(UserNotFoundException.class)
    public ProblemDetail missing(UserNotFoundException error) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, error.getMessage());
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail invalid(IllegalArgumentException error) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, error.getMessage());
    }
}
```

HTTP mapping annotations bind methods/paths. `@RequestBody` decodes JSON; `@Valid` rejects a blank name before calling the service. Creating returns 201 with a Location header; deleting returns 204 with no body. `@RestControllerAdvice` maps relevant exceptions to structured HTTP errors. Bean-validation failures are handled by MVC's validation exception handling and return 400; they do not use the IllegalArgumentException handler.

This distinction is what controller tests should verify: mapping, input conversion/validation, response serialization/status, and exception mapping. Business branches belong mainly in service tests.

F. MockMvc controller slice tests

UserControllerTest.java:

```
package com.example.testing;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    MockMvc mvc;

    @MockitoBean
    UserService service;

    @Test
    void getReturnsJson() throws Exception {
        when(service.getUserById(1)).thenReturn(new User(1, "Ravi"));
        mvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Ravi"));
    }

    @Test
    void missingUserReturns404() throws Exception {
        when(service.getUserById(99)).thenThrow(new UserNotFoundException(99));
```

```
        mvc.perform(get("/users/99"))
            .andExpect(status().isNotFound());
    }

    @Test
    void postReturnsCreatedAndLocation() throws Exception {
        when(service.createUser("Ravi")).thenReturn(new User(10, "Ravi"));
        mvc.perform(post("/users").contentType(MediaType.APPLICATION_JSON)
                .content("{\"name\":\"Ravi\"}"))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location", "/users/10"))
            .andExpect(jsonPath("$.id").value(10));
    }

    @Test
    void blankNameReturns400BeforeService() throws Exception {
        mvc.perform(post("/users").contentType(MediaType.APPLICATION_JSON)
                .content("{\"name\":\"   \"}"))
            .andExpect(status().isBadRequest());
        verifyNoInteractions(service);
    }

    @Test
    void deleteReturnsNoContent() throws Exception {
        mvc.perform(delete("/users/1"))
            .andExpect(status().isNoContent())
            .andExpect(content().string(""));
        verify(service).deleteUser(1);
    }
}
```

`@WebMvcTest(UserController.class)` loads MVC-related infrastructure and this controller, not the full service/repository graph. Relevant controller advice is included by slice scanning. `@Autowired` supplies MockMvc from the test context. `@MockitoBean` supplies a mock UserService **inside that Spring context**, allowing constructor injection into the real controller.

`mvc.perform` goes through MVC dispatch without opening a network socket. `andExpect` checks observable HTTP behavior. `jsonPath` checks fields in the serialized response. Request content type tells Spring how to decode the body. Blank input fails validation before service invocation; this is a useful boundary test rather than a duplicate service test.

Current equivalent: Spring Framework's `@MockitoBean` is the modern replacement for Boot's `@MockBean` in this baseline. Boot deprecated `@MockBean` starting with 3.4, for removal in 4.0. Its import is `org.springframework.test.context.bean.override.mockito.MockitoBean`. Older Boot/Framework combinations may need `@MockBean`; do not paste `@MockitoBean` into a release that lacks it. MockitoBean, MockBean deprecation.

This application has no Security starter. If your real application has security filters, slice tests must exercise the intended authentication/CSRF rules or explicitly choose a narrower test boundary. A 401/403 does not necessarily mean the controller mapping is broken. Do not disable security merely to make an authorization test pass.

G. Real in-memory repository and context integration test

`InMemoryUserRepository.java`:

```
package com.example.testing;

import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import org.springframework.stereotype.Repository;

@Repository
public class InMemoryUserRepository implements UserRepository {
```

```
    private final ConcurrentHashMap<Long, User> users = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong();

    @Override
    public Optional<User> findById(long id) { return Optional.ofNullable(users.get(id)); }

    @Override
    public User save(User user) {
        long id = user.id() == 0 ? sequence.incrementAndGet() : user.id();
        var saved = new User(id, user.name());
        users.put(id, saved);
        return saved;
    }

    @Override
    public boolean existsById(long id) { return users.containsKey(id); }

    @Override
    public void deleteById(long id) { users.remove(id); }
}
```

`@Repository` makes the real implementation injectable. Its concurrent map stores users for the process lifetime and its atomic sequence assigns demo IDs. It does not persist across restart, provide database transactions, or make the service's existence-check-plus-delete atomic. This implementation deliberately limits the integration-test boundary to MVC + Spring wiring + service + local storage.

`UserFlowIntegrationTest.java`:

```
package com.example.testing;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
class UserFlowIntegrationTest {
    @Autowired MockMvc mvc;
    @Autowired ObjectMapper json;

    @Test
    void createReadDeleteRoundTrip() throws Exception {
        var result = mvc.perform(post("/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"name\":\"  Ravi  \"}"))
            .andExpect(status().isCreated())
            .andReturn();

        long id = json.readTree(result.getResponse().getContentAsString()).get("id").asLong();
        try {
            mvc.perform(get("/users/{id}", id))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.name").value("Ravi"));
            mvc.perform(delete("/users/{id}", id))
                .andExpect(status().isNoContent());
            mvc.perform(get("/users/{id}", id))
                .andExpect(status().isNotFound());
        } finally {
            // Best-effort cleanup if an earlier assertion fails.
            mvc.perform(delete("/users/{id}", id));
        }
    }
}
```

`@SpringBootTest` loads the full application configuration; its default web environment does **not** start a listening server. `@AutoConfigureMockMvc` provides MVC request testing through the configured context. There are no service/repository mocks in this case. It creates a user, reads the returned generated ID, checks normalization, deletes, and verifies the user is gone.

This verifies component cooperation and context wiring. It is **not** a real-database integration test or a real-network end-to-end test. To include those boundaries, select actual database infrastructure or `@SpringBootTest(webEnvironment=RANDOM_PORT)` with a real HTTP client respectively. A transactional test rollback does not automatically roll back server work performed on a separate thread in a real-port test. Boot test environments.

H. Run and interpret

```
mvn test
```

Maven compiles production/test sources and runs the discovered tests. Expected here: all enabled cases pass and the educational disabled calculator case is reported as skipped. Read the assertion failure's expected/actual values and stack trace. Reports appear under `target/surefire-reports`.

To run only service tests: `mvn '-Dtest=UserServiceTest' test`. To run one method: `mvn '-Dtest=UserServiceTest#missingUserThrows' test`. Quoting is helpful in PowerShell, especially with `#`.

Choose tests from behavior: success, missing input, invalid input, failed dependency, and relevant boundaries. Use controlled time and deterministic data when a method depends on clocks/randomness. Avoid sleeps to wait for async work; use bounded condition/event synchronization. Coverage measures executed code, not the strength of assertions.

Important Interview Questions

10 beginner questions

1. **What is JUnit?** A Java framework for discovering/running tests, lifecycle hooks, assertions, and result reporting.
2. **What is a unit test?** A focused check of a small unit with controlled dependencies.
3. **What is an integration test?** A check of cooperating components at a stated boundary, such as service plus real repository.
4. **Why automate tests?** Repeatable regression feedback reduces manual effort and supports safer changes.
5. **What does @Test do?** Marks a method for Jupiter to execute as a test.
6. **What does @BeforeEach do?** Sets up conditions before each test invocation.
7. **@BeforeAll vs @BeforeEach?** Once per class versus once per invocation; BeforeAll is usually static in the default lifecycle.
8. **What is an assertion?** A check that actual behavior meets an expected condition.
9. **How do you test an exception?** Use assertThrows with a lambda that invokes the failing operation.
10. **What is a parameterized test?** One test logic executed with multiple supplied argument sets.

10 intermediate questions

1. **JUnit vs Mockito?** JUnit runs/asserts tests; Mockito creates and controls dependencies and checks their interactions.
2. **@Mock vs @MockitoBean?** Mock creates a Mockito field; MockitoBean adds/replaces a dependency in a Spring test context.
3. **What does @InjectMocks do?** Builds the object under test and injects available Mockito mocks; it is not Spring context wiring.

4. **when/thenReturn vs verify?** Stubbing controls a dependency's response; verification checks important calls occurred.
5. **@SpringBootTest vs @WebMvcTest?** Full Boot context versus an MVC-focused slice; neither inherently implies a listening HTTP server.
6. **What does MockMvc test?** MVC dispatch, validation, serialization, status, and filters using mock requests, without a real network connection.
7. **Why use assertAll?** Report several independent assertion failures in one group instead of stopping at the first one.
8. **Why must tests be independent?** Order-dependent or shared-state tests fail unpredictably and hide the actual cause.
9. **Should every service test load Spring?** No. Constructor-injected business behavior can usually be tested directly with controlled dependencies.
10. **Is 100% coverage enough?** No. Executed lines may have weak assertions, missing cases, or untested external boundaries.

5 scenario-based questions

1. **A mocked repository field is null. Why?** Mockito annotations were not initialized, or the test setup is wrong; use MockitoExtension or explicit mocks.
2. **WebMvcTest cannot create UserController. What do you check?** Supply its service dependency with MockitoBean and verify the selected controller/configuration.
3. **A controller test returns 403 for POST. What may cause it?** Spring Security CSRF/authorization filters; include valid test security context and CSRF where required.
4. **Service unit tests pass but production DB query fails. Why?** The repository was mocked; add integration coverage for real persistence behavior.
5. **Tests pass alone but fail together. What do you investigate?** Shared state, context-cache fixtures, leaked resources, fixed IDs/ports, time assumptions, and execution-order dependence.

Common Mistakes

- **Wrong imports:** Jupiter uses `org.junit.jupiter`, not JUnit 4 annotation packages.
- **Mockito fields without extension:** initialize them explicitly or use MockitoExtension.
- **Mock the class under test:** keep the service real and mock its dependencies.
- **Load a full context unnecessarily:** choose direct unit tests or a suitable slice when sufficient.
- **Only check status 200:** assert important body fields, errors, and effects too.
- **Expect MockMvc to validate a real server/network:** add a real-port test when that boundary matters.
- **Assume @SpringBootTest means no mocks:** inspect the test's actual bean replacements and infrastructure.
- **Use deprecated @MockBean blindly:** use MockitoBean when supported; keep version-specific imports consistent.
- **Disable failing tests:** investigate the failure; disabled tests do not supply coverage.
- **Brittle interaction verification:** check meaningful effects rather than every implementation detail.
- **Uncontrolled data/time/sleeps:** use fresh fixtures, explicit cleanup, controlled clocks, and bounded synchronization.
- **Assume rollback covers remote server transactions:** recognize thread and transaction boundaries.

Quick Revision

- Arrange → Act → Assert.
- Unit tests isolate behavior; integration tests verify cooperation at an explicit boundary.
- Jupiter annotations define tests/lifecycle; assertions define expectations.
- Mockito supplies mocks/stubs/verifications; it does not replace JUnit.
- Mock and InjectMocks work outside Spring; MockitoBean works in a Spring test context.
- WebMvcTest narrows configuration; SpringBootTest loads full Boot configuration.
- MockMvc has no listening server; RANDOM_PORT has a real one.
- Test meaningful success, invalid, missing, and failure cases; coverage alone is insufficient.

Remember This for Interviews

1. “A useful test checks behavior and a stated boundary, not just an annotation.”
2. “I test service logic without Spring when constructor injection makes that possible.”
3. “JUnit executes and asserts; Mockito controls and observes dependencies.”
4. “Mocking the object under test bypasses the code I want to verify.”
5. “A WebMvc slice proves HTTP behavior with controlled service responses.”
6. “A full application context is not automatically a real-network or real-database test.”
7. “Independent fixtures and explicit cleanup keep tests deterministic.”
8. “High coverage is useful feedback, not proof that the assertions or requirements are complete.”

Final Comparison Tables

Spring Cloud Gateway vs Load Balancer

Dimension	Spring Cloud Gateway	Load balancer
Main responsibility	API routing and edge policy	Distribute traffic among eligible targets
Typical decisions	Path/header/method, authentication, transformations	Instance health and selection policy
Application awareness	Custom filters and API-specific behavior	Depends on implementation; L7 products also inspect HTTP
Relationship	Can incorporate service-instance balancing	Can distribute incoming traffic across gateway instances
Example	`/api/users` → User Service	Choose User instance A or B

A product can combine both roles. “Gateway handles APIs, load balancer chooses targets” describes responsibilities, not a hard feature boundary.

Spring Cloud Gateway vs Nginx / Traditional Reverse Proxy

Dimension	Spring Cloud Gateway	Nginx
Foundation	Spring application; WebFlux or MVC variant	General web server/reverse proxy
Configuration/customization	YAML and Java/Spring extension points	Nginx configuration and supported modules/extensions
Common strengths	Spring-integrated discovery/security/custom filters	General HTTP proxying, TLS, static serving, balancing
Overlap	Reverse proxying, routing, limits, and policies	Reverse proxying, routing, limits, and policies
Good fit	Java team needs API-specific Spring behavior	Infrastructure needs a general proxy/web server
Performance	Depends on processing, workload, tuning, resources	Depends on processing, workload, tuning, resources

An API gateway can be a specialized reverse proxy; these are overlapping concepts. Do not claim Nginx cannot authenticate/limit/balance or that one is universally faster. They may be used together. Nginx capabilities.

Synchronous vs Event-Driven Architecture

Dimension	Synchronous interaction	Event-driven interaction
Trigger	Request to a specific operation	Publication of a fact
Completion	Caller usually waits for response	Subscriber processing occurs independently
Coupling	API contract and runtime availability of callee	Event contract, broker, and delivery behavior

Dimension	Synchronous interaction	Event-driven interaction
Failure visibility	Usually immediate to caller	Often visible through retries/status/failure events
Consistency	Immediate for that call; not automatically global	Commonly eventual across consumers
Buffer/replay	Must be designed separately	Often supported by transport/storage such as Kafka
Suitable example	Fetch current product details	React independently to OrderCreated

Kafka Producer vs Consumer

Dimension	Producer	Consumer
Operation	Publishes records	Reads/processes records
Encoding	Serializes key and value to bytes	Deserializes bytes to usable values
Routing/progress	Topic, key, partition choice	Group, partition assignment, offsets
Success boundary	Broker-send result/ACK	Processing/recovery and offset progress
Main risks	Ambiguous send failures, duplicate business publication	Redelivery, lag, poison records, repeated effects
Spring API	KafkaTemplate	KafkaListener and listener container

Tailwind CSS vs Normal CSS

Dimension	Tailwind	Normal/custom CSS
Authoring	Compose utilities in markup	Write selectors and declarations
Design consistency	Theme tokens and utility scales	Team-defined variables/conventions
Readability tradeoff	Styling local to markup; more classes	Cleaner markup; follow selectors to see style
Build needs	Tailwind compilation/source detection	Depends on chosen tooling; plain CSS needs no framework compiler
Flexibility	Utilities plus arbitrary/custom CSS	Direct full CSS control
Reuse	Components/patterns	Classes/components/stylesheets

Tailwind CSS vs Bootstrap

Dimension	Tailwind	Bootstrap
Primary approach	Utility-first design primitives	Styled components, layout and utilities
Starting visual appearance	Compose your design	Established component defaults
Custom design	Fine-grained composition	Override/theme established styles
Interaction behavior	CSS framework; you supply behavior	Optional JS component behavior is available
Best-fit tendency	Custom product interfaces	Rapid assembly around established UI conventions
Accessibility	Still requires correct semantics/interaction checks	Still requires correct semantics/interaction checks

Bootstrap supplies optional JavaScript components in addition to its CSS. Bootstrap introduction.

REST vs WebSocket

Dimension	REST-style HTTP API	WebSocket
Nature	API architectural style commonly using HTTP	Transport protocol
Interaction	Request/response	Persistent bidirectional messages
Natural use	Fetch/create/update resources	Frequent interactive updates
HTTP semantics	Methods, status codes, resource URLs	Define application message semantics separately
Connection state	Requests need not depend on a session	Active sessions/subscriptions often matter
Recovery	Retry/resource reads according to semantics	Reconnect and restore messaging state

These are not identical categories. A system may use both REST and WebSocket.

WebSocket vs SSE

Dimension	WebSocket	SSE
Direction	Both directions on the transport	Server to client; requests handle the other direction
Data	Text or binary messages	Text event stream
Browser API	WebSocket; application reconnection logic	EventSource with browser reconnect support
Protocol path	WebSocket negotiation and message frames	Long-lived HTTP response
Resume history	Application design required	Last-Event-ID can help when the server implements history/resume
Natural use	Chat/collaborative bidirectional interaction	Feeds, progress, one-way notifications

Raw WebSocket vs STOMP over WebSocket

Dimension	Raw WebSocket	STOMP over WebSocket
Layer	Message transport	Messaging protocol carried over transport
Commands/destinations	Define your own	Standard SEND/SUBSCRIBE/MESSAGE conventions
Subscription model	Implement yourself	Supported by STOMP stack/broker
Overhead	Can use compact custom messages	STOMP commands/headers add overhead
Spring integration	WebSocket handlers	MessageMapping and broker integration
Guarantee	No inherent durable business delivery	Still depends on broker, acknowledgement and application design

JUnit vs Mockito

Dimension	JUnit	Mockito
Purpose	Run tests, lifecycle, assertions, reporting	Replace/control dependencies and verify calls
Main examples	Test, BeforeEach, assertEquals	Mock, InjectMocks, when, verify
Can work alone?	Yes, for pure Java tests	Used with a runner/framework or directly, but does not supply a complete test suite runner
Together	JUnit runs a case using Mockito-controlled dependencies	Mockito supplies isolation while JUnit checks outcomes

Unit Test vs Integration Test

Dimension	Unit	Integration
Boundary	One small unit with controlled dependencies	Cooperating real components/infrastructure
Spring required?	Usually no	Depends on the integration boundary
Typical speed	Fast	Often slower
Failure diagnosis	Narrower	May span wiring, configuration, and interaction
Proves	Local decisions/results/effects	Components work together at the tested boundary
Cannot automatically prove	Real repository/network behavior when mocked	Untested environments or end-to-end behavior outside the boundary

Spring Test Annotation Choice

Tool / annotation	Loads Spring context?	Primary purpose
Jupiter + MockitoExtension	No	Isolated service behavior
Mock / InjectMocks	No by themselves	Mockito dependency setup
WebMvcTest	MVC slice	Controller HTTP behavior
MockitoBean	Works within a Spring test context	Supply/replace a mocked context dependency
SpringBootTest + AutoConfigureMockMvc	Full Boot context, no listening server by default	MVC + application bean cooperation
SpringBootTest(RANDOM_PORT)	Full context with actual server	Real HTTP/network boundary

Practice Checklist Within These Five Topics

- Rebuild the two Gateway routes from memory; explain why matching and forwarded paths differ.
- Publish an order event, identify each consumer group, and describe every crash boundary.
- Build the card/form/grid using Tailwind, then explain each utility and breakpoint without looking it up.
- Trace chat from handshake to SEND, Java handler, broker, and subscriber; explain reconnect and missed-message limits.
- Write service success/error unit tests and a MockMvc validation test; state exactly what each test proves.

When answering an interview question, use **definition → purpose → concrete example → limitation**. For scenarios, first identify the failing boundary, then explain the evidence to inspect and the recovery policy.